# UE23CS352B - Object Oriented Analysis & Design

## Mini Project Report

## Title: Hotel Management System

***Submitted by:***

***Team Members***

| NAME | SRN |
|---|---|
| AARUSH RYAN LOBO | PES2UG23CS011 |
| ABHIN G DAS | PES2UG23CS019 |
| AMOGH VAIDYA | PES2UG23CS056 |
| KUSUMITA | PES2UG22CS278 |

Semester: 6 - Section: A

**Faculty Name: Ms. Senthil Anandhi**

**January - May 2026**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**
FACULTY OF ENGINEERING
**PES UNIVERSITY**
(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru – 560 085, Karnataka, India

**Problem Statement:**

The Hotel Management System is a web-based application designed to digitize and streamline the day-to-day operations of a hotel. Traditional hotel management involves significant manual effort in handling guest registrations, room bookings, billing, housekeeping coordination, and service management. These manual processes are error-prone, slow, and difficult to scale.

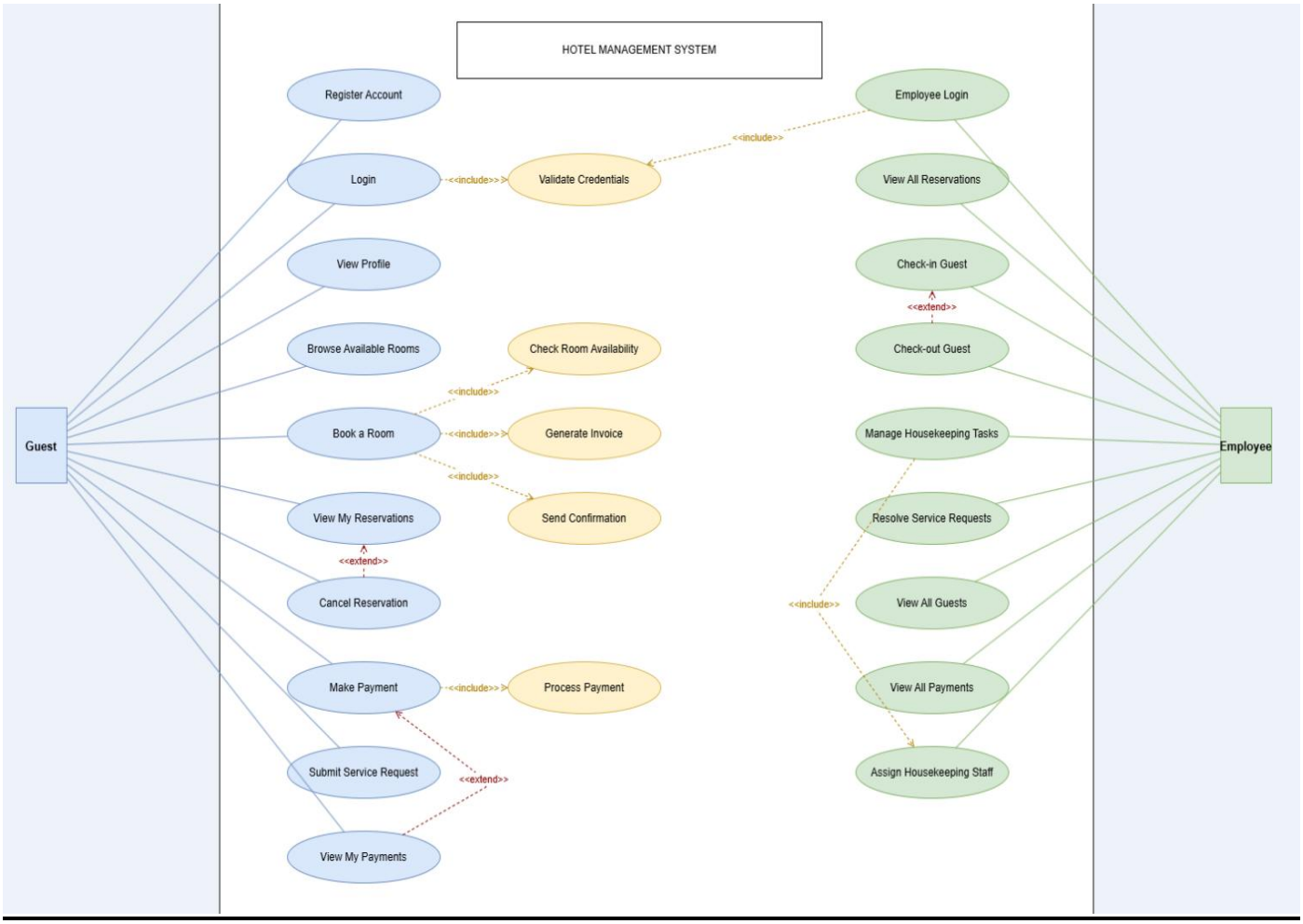
This system provides a centralized platform where guests can register, browse available rooms, make reservations, and process payments — all through a browser. Simultaneously, hotel employees can manage reservations, oversee housekeeping tasks, resolve service requests, and track billing from a dedicated employee dashboard.

**Key Features:**

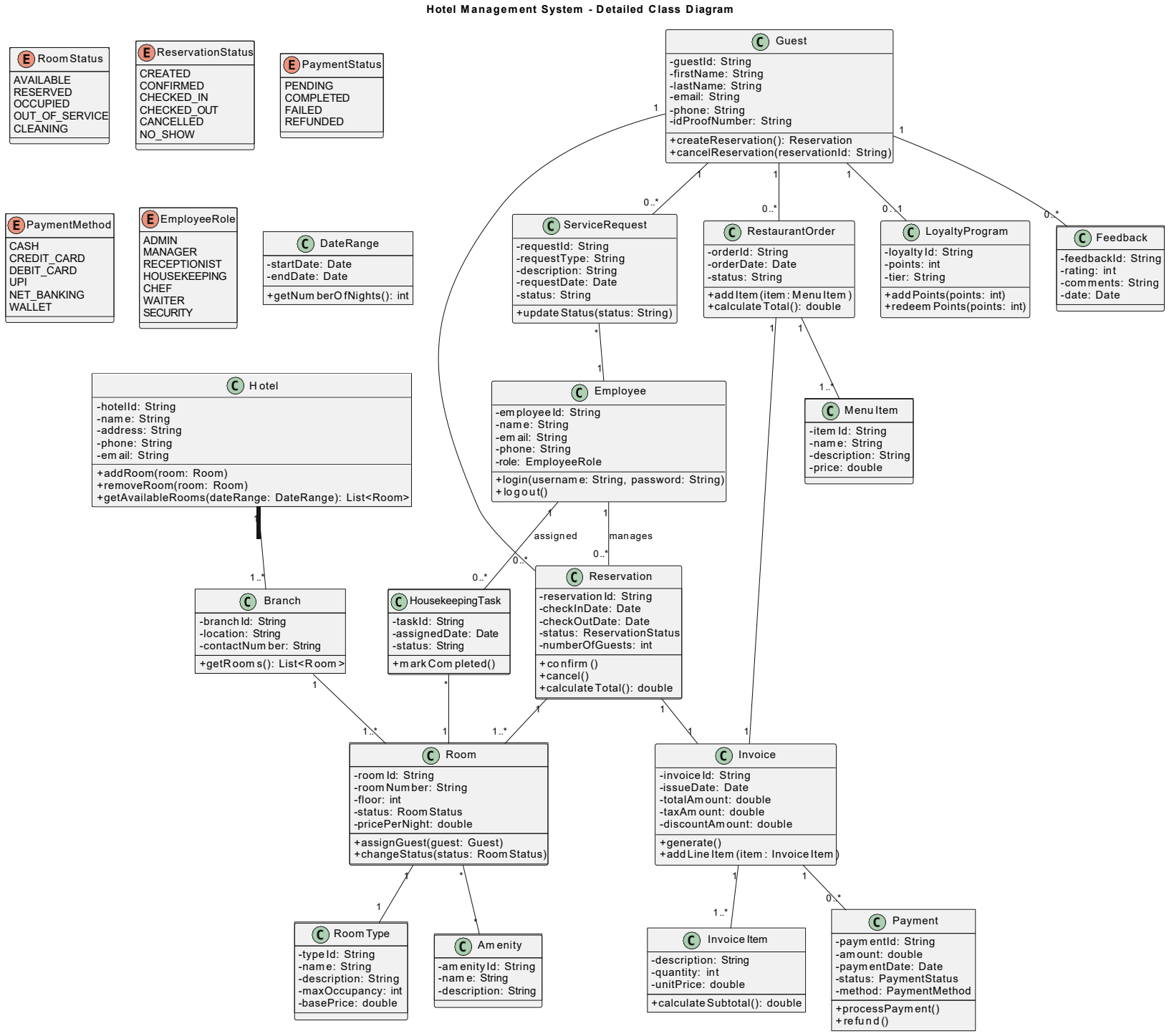
- Guest registration, login, and profile management
- Room browsing and availability checking
- Room reservation with check-in and check-out date selection
- Automatic invoice generation with 18% GST calculation
- Multiple payment method support: Cash, Credit Card, Debit Card, UPI, Wallet
- Service request submission and tracking by guests
- Housekeeping task assignment and completion tracking by employees
- Employee dashboard with reservation management, check-in/check-out operations
- Role-based access control for guests and employees
- Persistent data storage using MySQL database

Models:

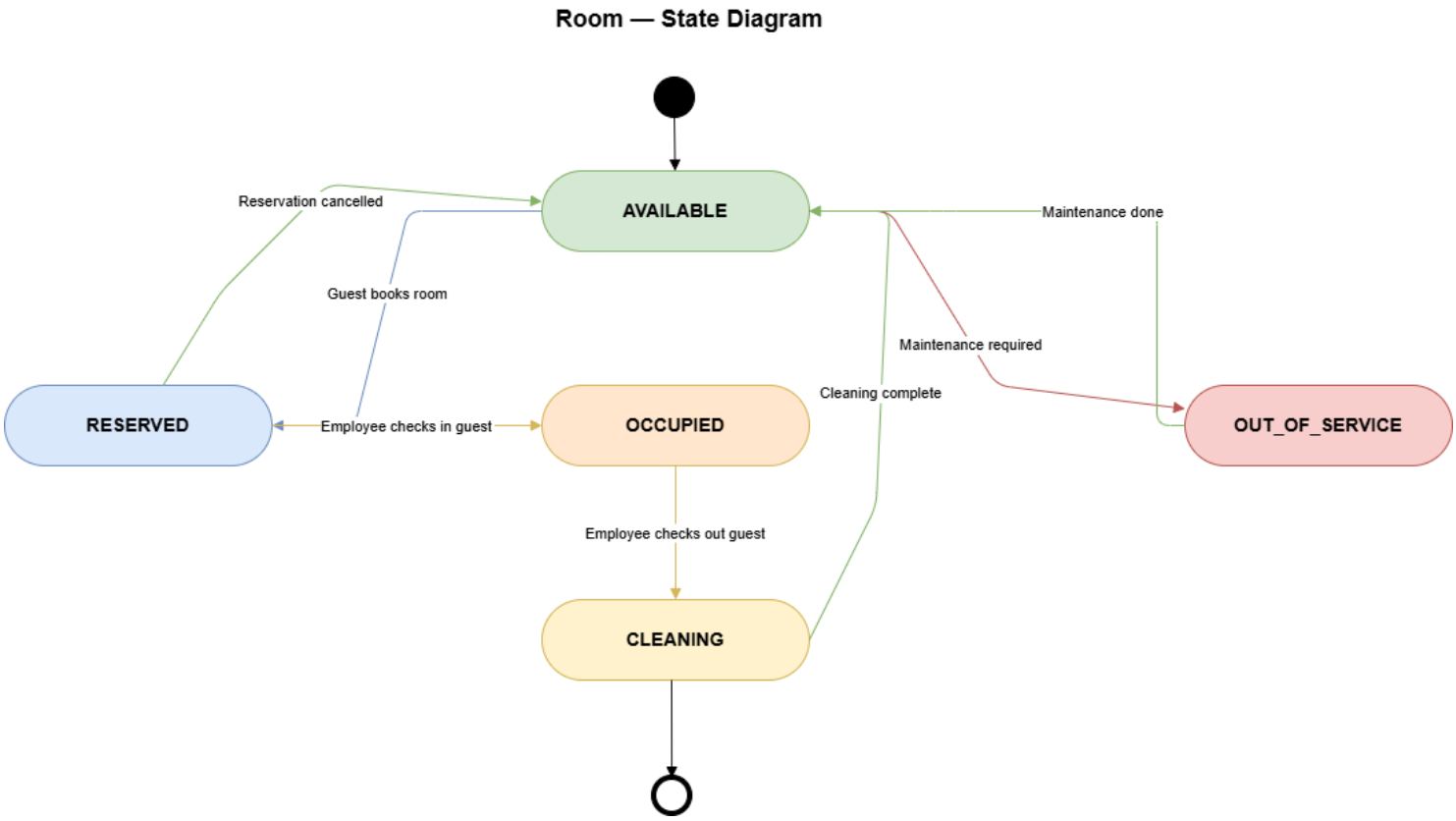
Use Case Diagram:

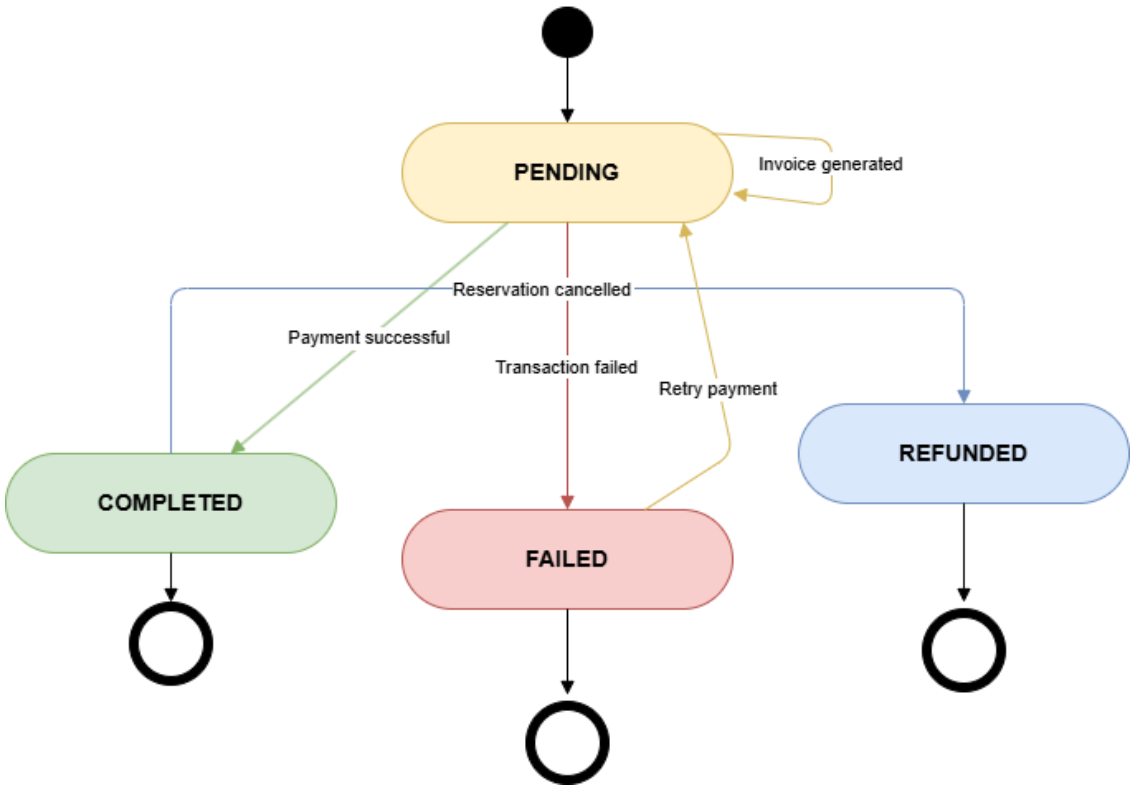


# Class Diagram:

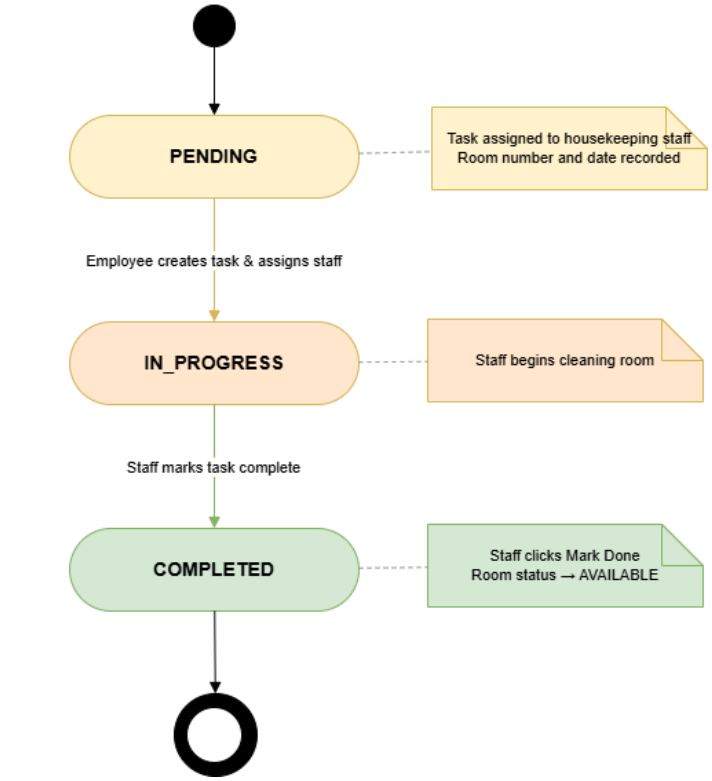


## State Diagram:

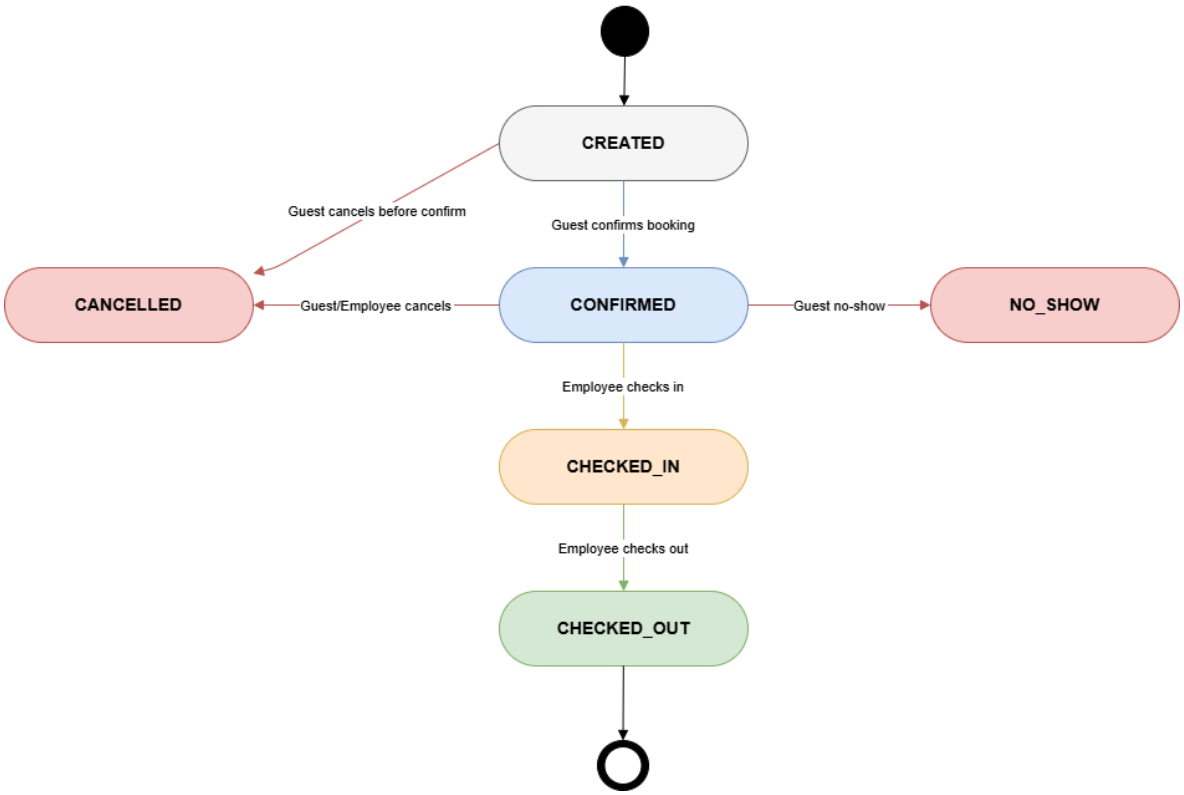


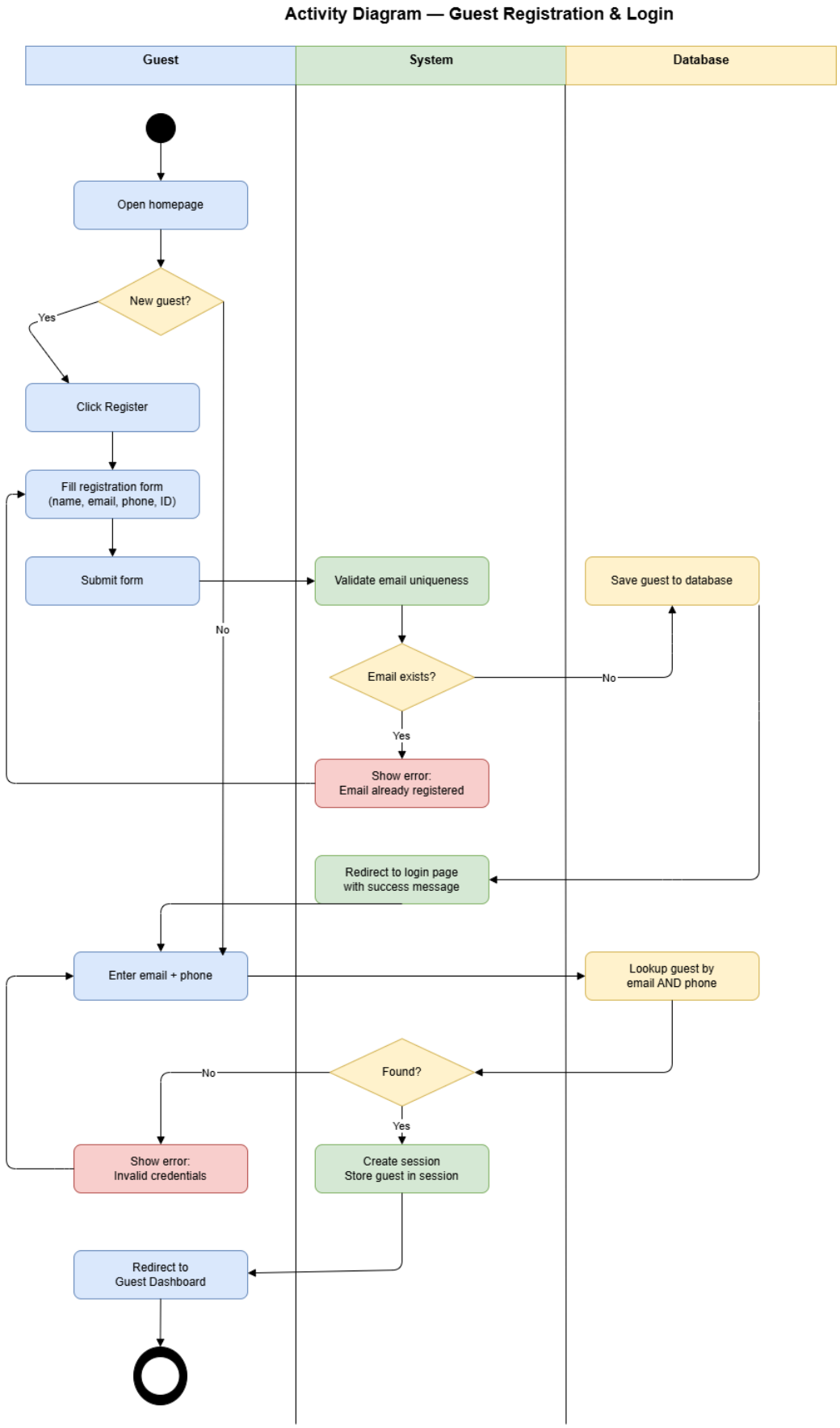
## Housekeeping Task — State Diagram



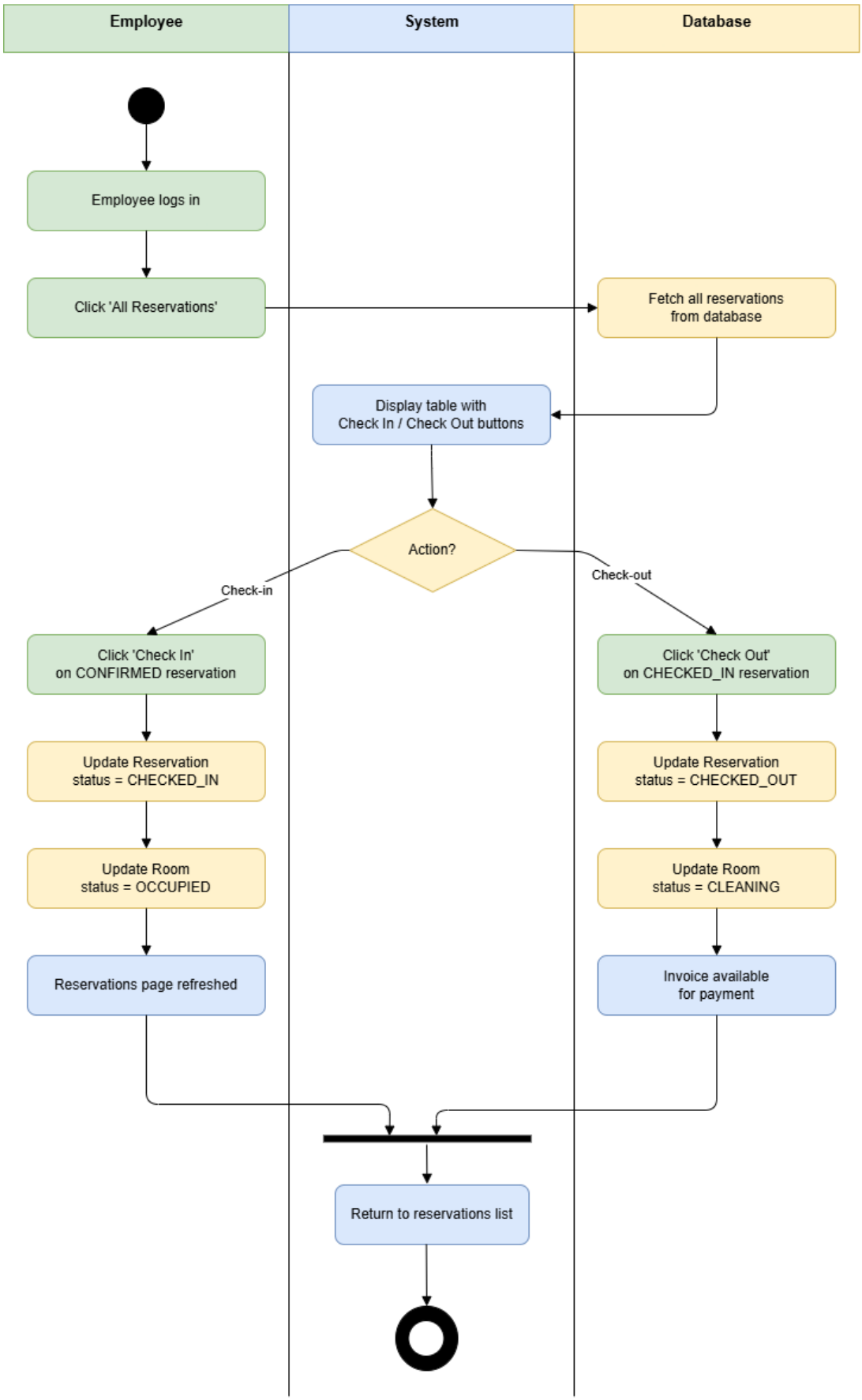

## Reservation — State Diagram



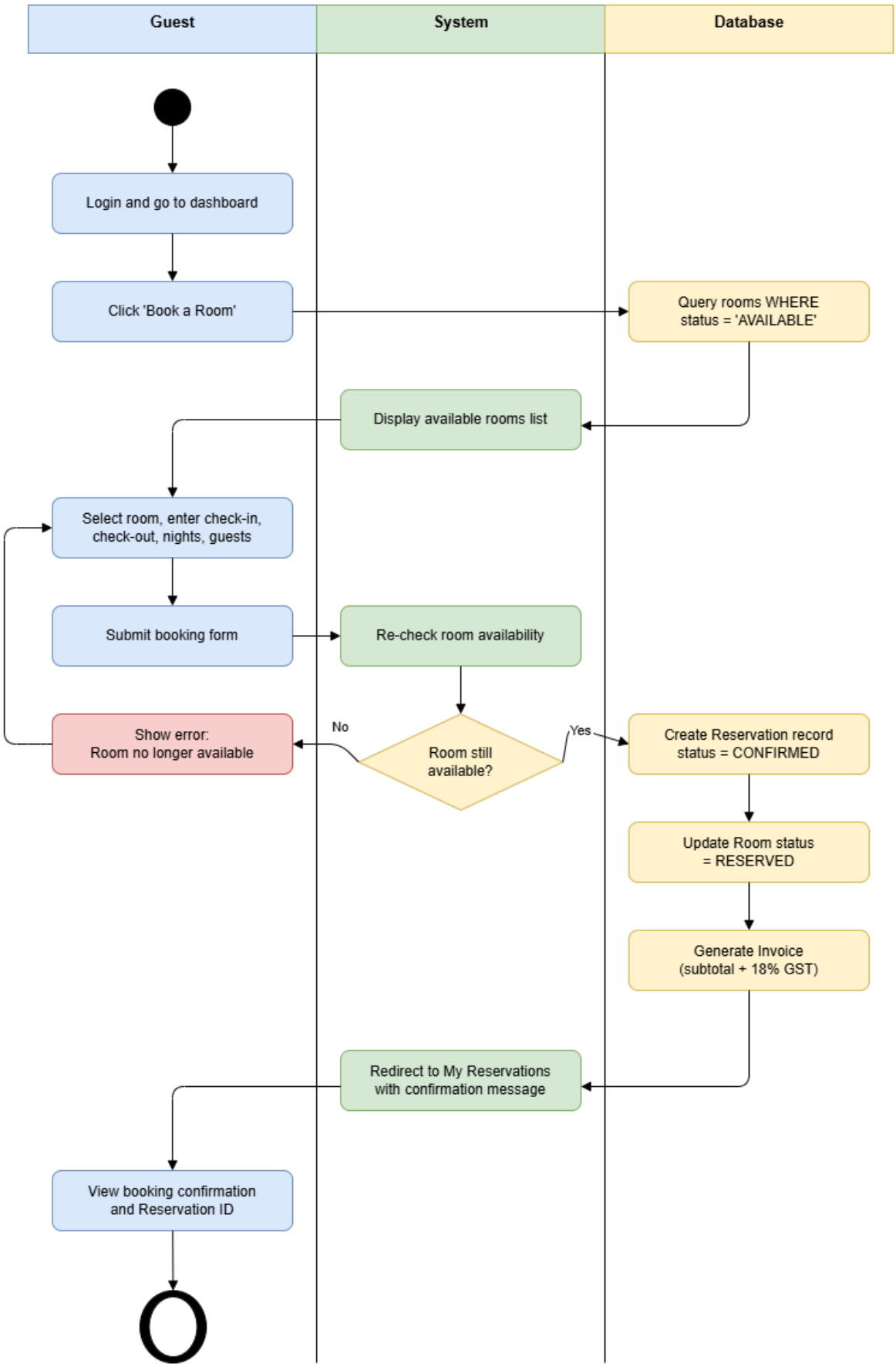
## Activity Diagrams:



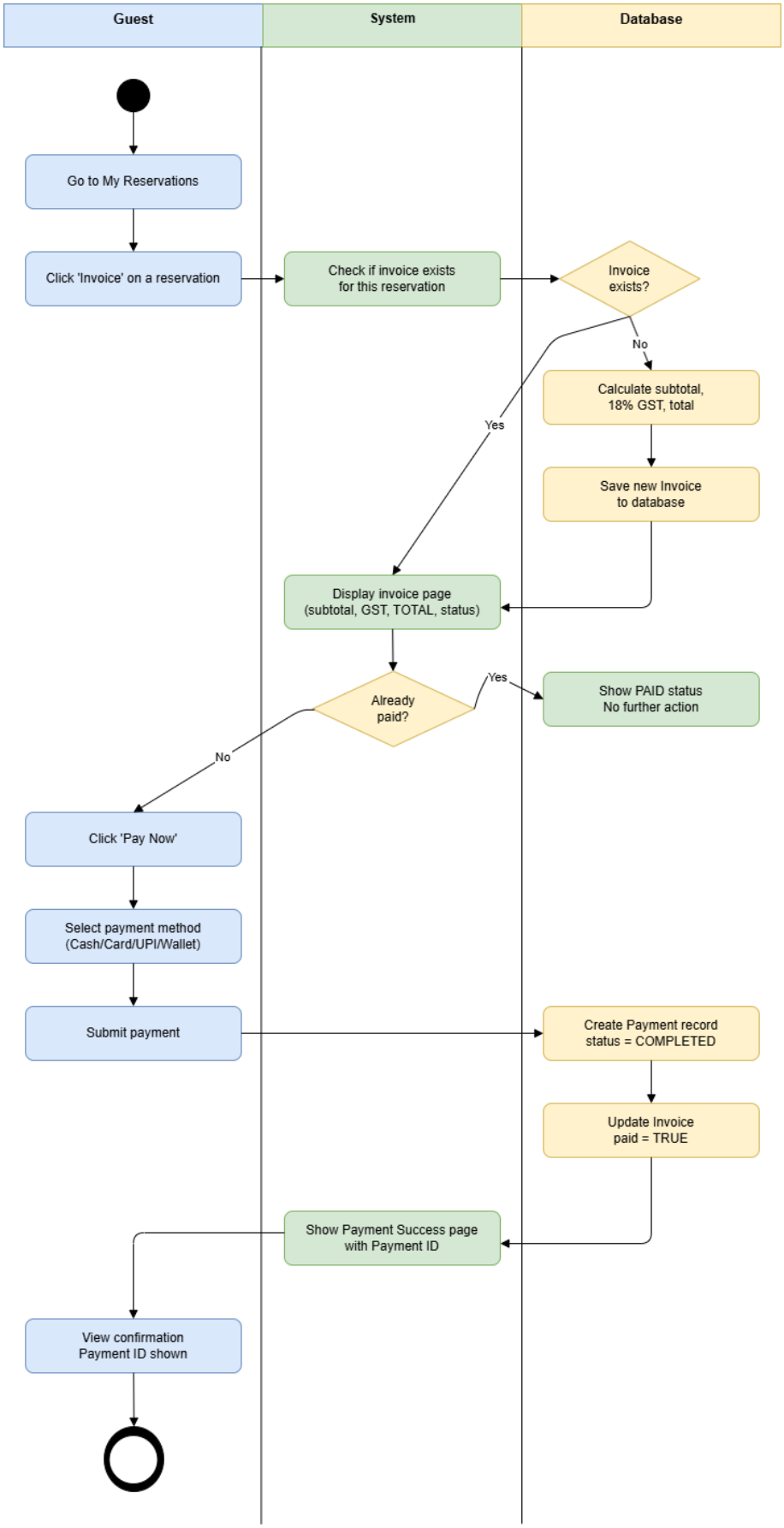
## Activity Diagram — Check-in & Check-out



## Activity Diagram — Room Booking



## Activity Diagram — Payment Process



**Design Principles, and Design Patterns:**

## MVC Architecture used:

The Hotel Management System follows the Model-View-Controller (MVC) architectural pattern, enforced by the Spring Boot framework.

| Layer | Technology Used | Role in the Project |
|---|---|---|
| Model | Java Entity Classes + JPA + MySQL | Represents data — Guest, Room, Reservation, Invoice, Payment, Employee, ServiceRequest, HousekeepingTask. Each class maps to a database table. |
| View | Thymeleaf HTML Templates | Renders the UI in the browser. Pages include guest login, registration, dashboard, booking form, invoice, payment, housekeeping, and service requests. |
| Controller | Spring MVC @Controller Classes | Handles HTTP requests, processes business logic via Service classes, and returns the correct View with Model data. Controllers: GuestController, ReservationController, PaymentController, ServicesController, EmployeeController. |

In Spring Boot, the @Controller annotation marks a class as the C in MVC. The @Service layer sits between Controller and Model, containing business logic. The Repository layer (extending JpaRepository) handles all database operations. This clean separation ensures each team member could independently develop their module without breaking others.

## Design Principles

The following four SOLID design principles are applied in the project, one owned by each team member.

### 4.1 Single Responsibility Principle (SRP) — Abhin G Das

Definition: A class should have only one reason to change — it should have only one job.

Application in the project:

- GuestController is responsible only for handling HTTP requests related to guests — login, register, dashboard, profile, logout.
- GuestService is responsible only for guest business logic — registering, authenticating, and retrieving guests.
- GuestRepository is responsible only for database operations on the guests table.
- These are three separate classes, each with a single responsibility. None of them handles invoice logic, room logic, or employee logic.

Without SRP: If GuestController also handled payments, a change in payment logic would force changes to the guest controller, breaking the guest module unexpectedly.

### 4.2 Open/Closed Principle (OCP) — Aarush Ryan Lobo

Definition: Software entities should be open for extension but closed for modification.

Application in the project:

- The ReservationService class is closed for modification — its core booking, check-in, and check-out logic does not need to change when new features are added.
- It is open for extension — if a new reservation type (e.g., group booking) is needed, a new subclass or method can be added without modifying the existing methods.
- The RoomRepository extends JpaRepository, which provides built-in CRUD operations. New query methods like findByStatus() were added without modifying the existing JpaRepository interface — this is OCP in action.

Without OCP: Any new feature would require editing existing working code, risking regression bugs.

**4.3 Liskov Substitution Principle (LSP) — Amogh Vaidya**

Definition: Objects of a subclass should be replaceable with objects of the parent class without breaking the application.

Application in the project:

- All Spring Data repositories (GuestRepository, RoomRepository, ReservationRepository, InvoiceRepository, PaymentRepository) extend JpaRepository.
- Anywhere that JpaRepository is expected, any of these specific repositories can be substituted without breaking the application — because they honor the contract defined by JpaRepository.
- For example, PaymentService uses InvoiceRepository and PaymentRepository. Both are JpaRepository subtypes and can be injected and used interchangeably for common operations like save(), findById(), findAll().

Without LSP: If a repository overrode a method in a way that violated the parent contract (e.g., save() not actually persisting), the application would silently fail.

**4.4 Dependency Inversion Principle (DIP) — Kusumita**

Definition: High-level modules should not depend on low-level modules. Both should depend on abstractions.

Application in the project:

- ServicesController (high-level) does not directly instantiate ServicesService or HousekeepingTaskRepository. It depends on abstractions injected by Spring via @Autowired.
- ServicesService depends on ServiceRequestRepository and HousekeepingTaskRepository interfaces, not on their concrete implementations.
- Spring Boot's dependency injection container manages the wiring — if the database implementation changes from MySQL to PostgreSQL, the service and controller code does not need to change.

Without DIP: ServicesController would do 'new ServicesService(new HousekeepingTaskRepository(...))' — tightly coupled, impossible to test independently.

## Design Patterns

The following four design patterns are implemented in the project — one creational, one structural, one behavioral, and one enforced by the framework.

### 5.1 Singleton Pattern (Creational) — Abhing Das

Intent: Ensure a class has only one instance and provide a global point of access to it.

Application in the project:

- Spring Boot by default creates all @Service and @Repository beans as Singletons — there is exactly one instance of GuestService, EmployeeService, PaymentService, etc. in the application context.
- This means all controllers share the same GuestService instance. There is no duplication of database connections or business logic objects.
- The Spring ApplicationContext itself acts as the Singleton container.

Code evidence: @Service annotation on GuestService, @Repository on GuestRepository — Spring manages these as singleton beans by default.

Without Singleton: Each controller would create its own service instance, leading to inconsistent state and wasted memory.

### 5.2 Facade Pattern (Structural) — Aarush Ryan Lobo

Intent: Provide a simplified interface to a complex subsystem.

Application in the project:

- ReservationService acts as a Facade over the complex subsystem of ReservationRepository, RoomRepository, and related business logic.
- When a guest books a room, the controller simply calls reservationService.bookRoom(...). Internally, this method: validates room availability, creates the reservation object, updates the room status, saves both to the database — all hidden behind one method call.
- The ReservationController does not need to know about RoomRepository or how room status is updated. It just calls the facade.

Code evidence: ReservationController calls reservationService.bookRoom(), reservationService.checkIn(), reservationService.checkOut() — all complex multi-step operations hidden behind simple method names.

### 5.3 Observer Pattern (Behavioral) — Kusumita

Intent: Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified automatically.

Application in the project:

- When a guest submits a ServiceRequest, the ServicesService saves it with status OPEN. The employee dashboard observes this by querying all open requests — effectively watching for state changes.
- Conceptually, the ServiceRequest entity is the subject. When its status changes from OPEN to RESOLVED by an employee, the guest's view of their requests is automatically updated on the next page refresh.
- Spring Data JPA's change detection acts as the notification mechanism — any save() triggers a database update which all observers (views) will see on next query.

Code evidence: ServicesController.resolve() calls servicesService.resolveRequest() which updates status to RESOLVED and saves — any view that subsequently calls getAllRequests() sees the updated state.

### 5.4 Repository Pattern (Framework-enforced) — Amogh Vaidya

Intent: Abstract the data layer, providing a collection-like interface for accessing domain objects.

Application in the project:

- Spring Data JPA enforces the Repository pattern through the JpaRepository interface.
- Each entity has a dedicated repository: GuestRepository, RoomRepository, ReservationRepository, InvoiceRepository, PaymentRepository, ServiceRequestRepository, HousekeepingTaskRepository, EmployeeRepository.
- Business logic in Service classes never writes raw SQL. They use repository methods: save(), findById(), findAll(), findByGuestId(), findByStatus() etc.
- Custom queries are added as method signatures — Spring generates the implementation automatically.

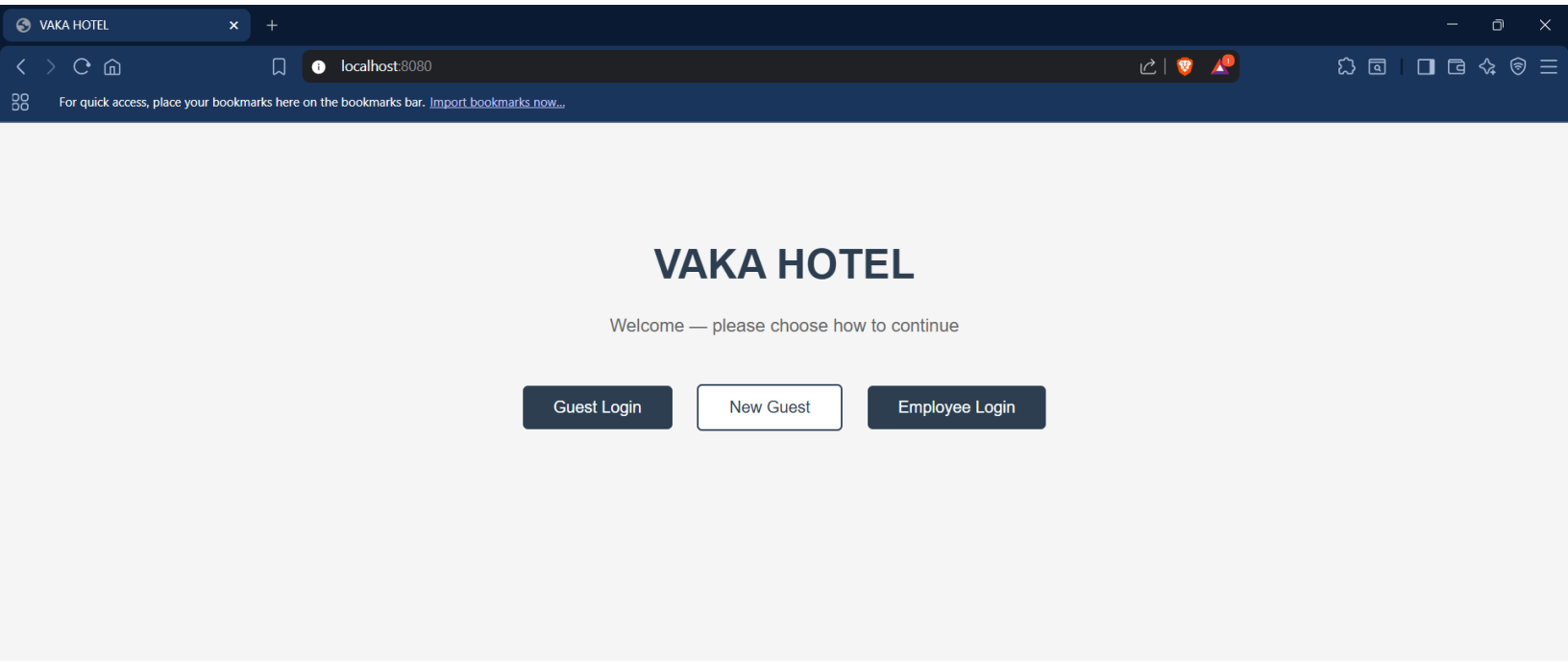
Code evidence: InvoiceRepository.findByReservationId(String) — declared as one line, Spring generates the full SQL query implementation at runtime.

**Github link to the Codebase: https://github.com/abhin012/OOAD_MINI**

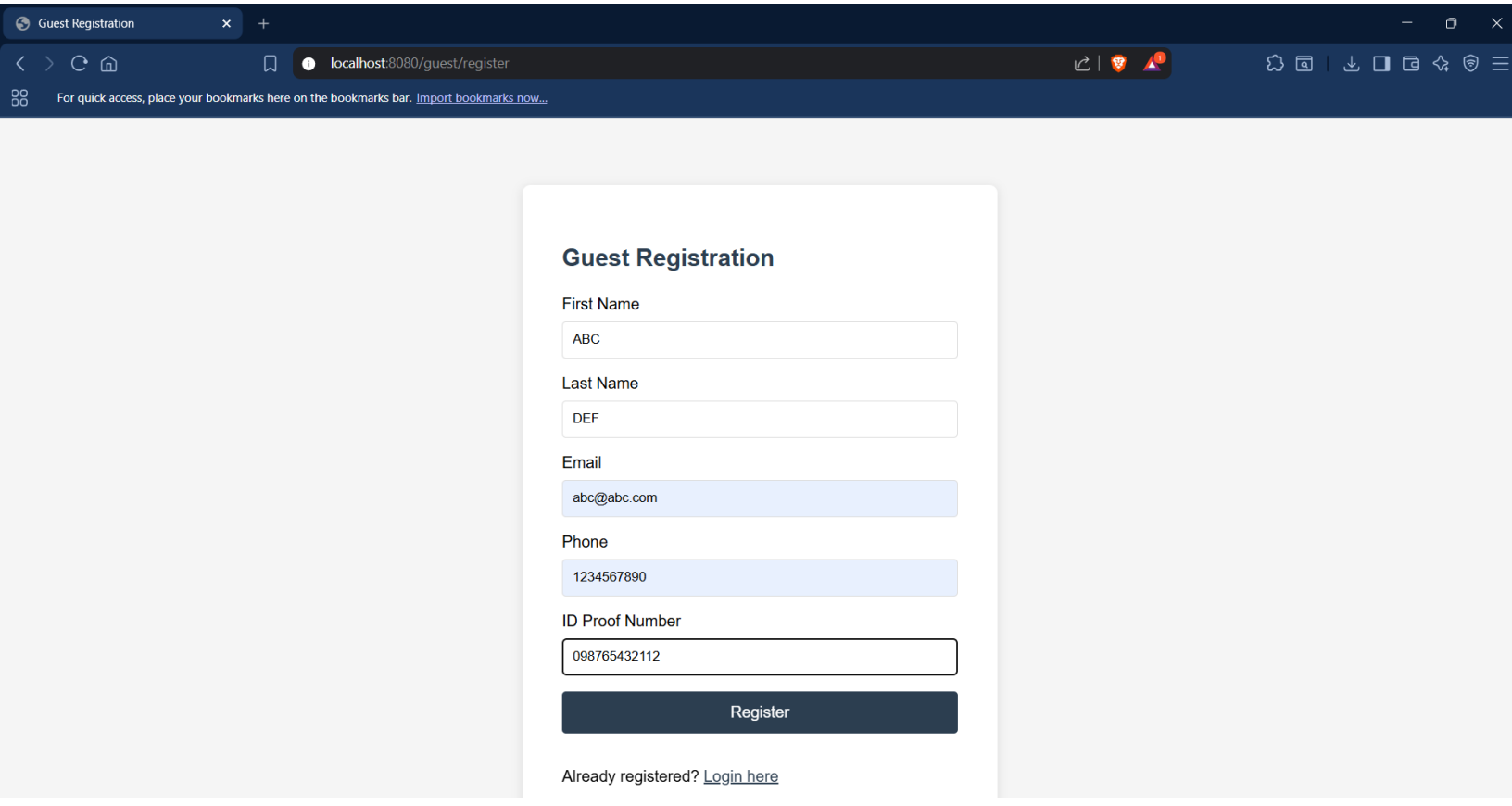
**Screenshots**

UI:

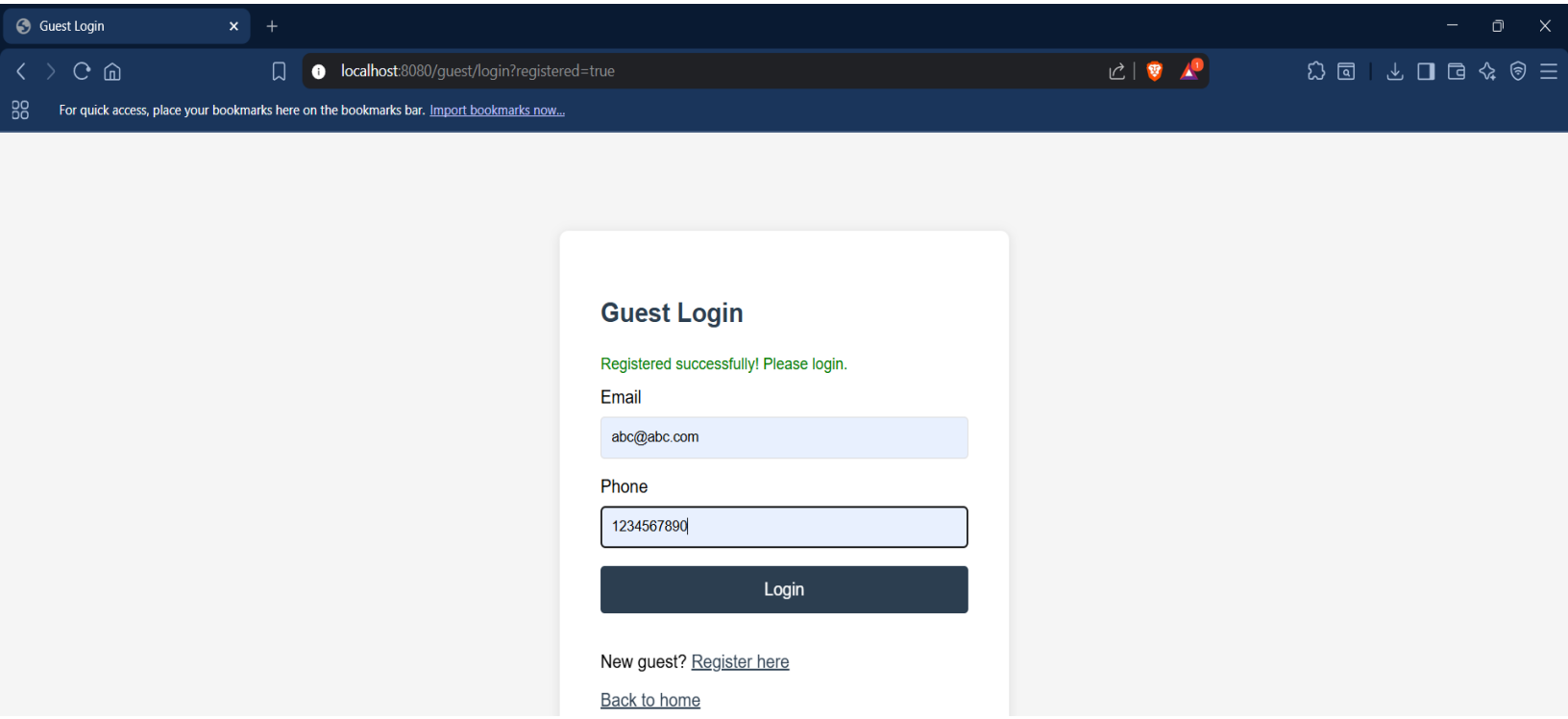
HOME PAGE:



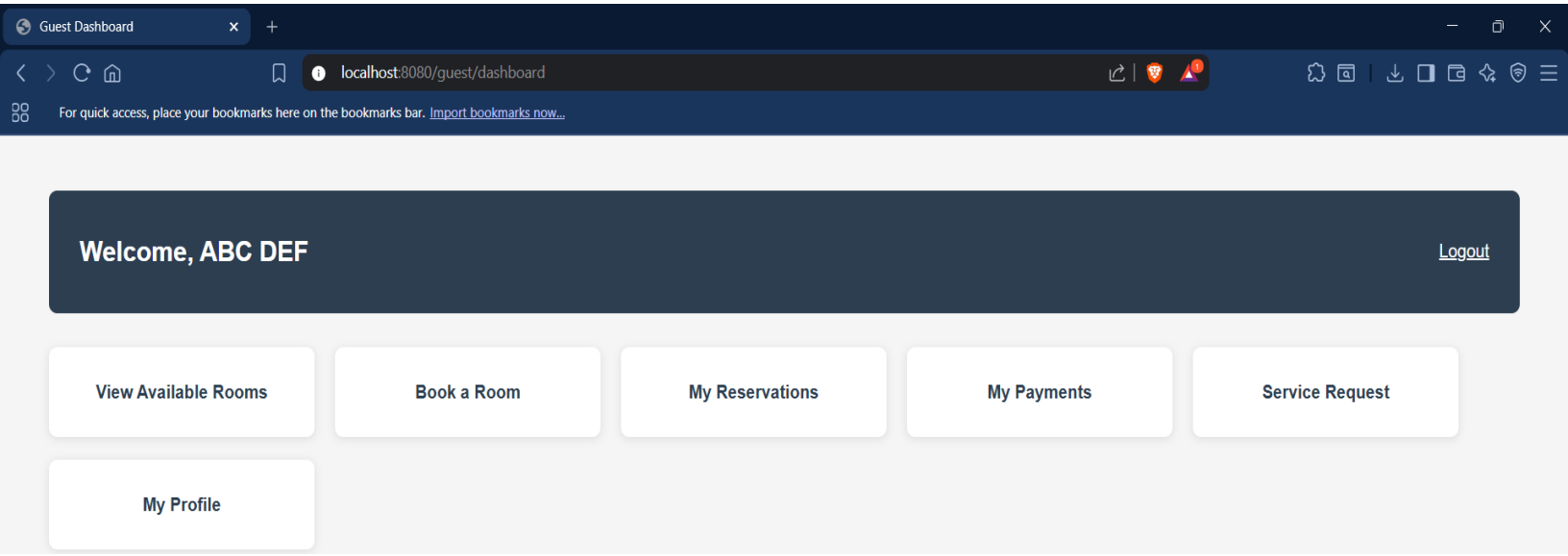

1. GUEST REGISTRATION:



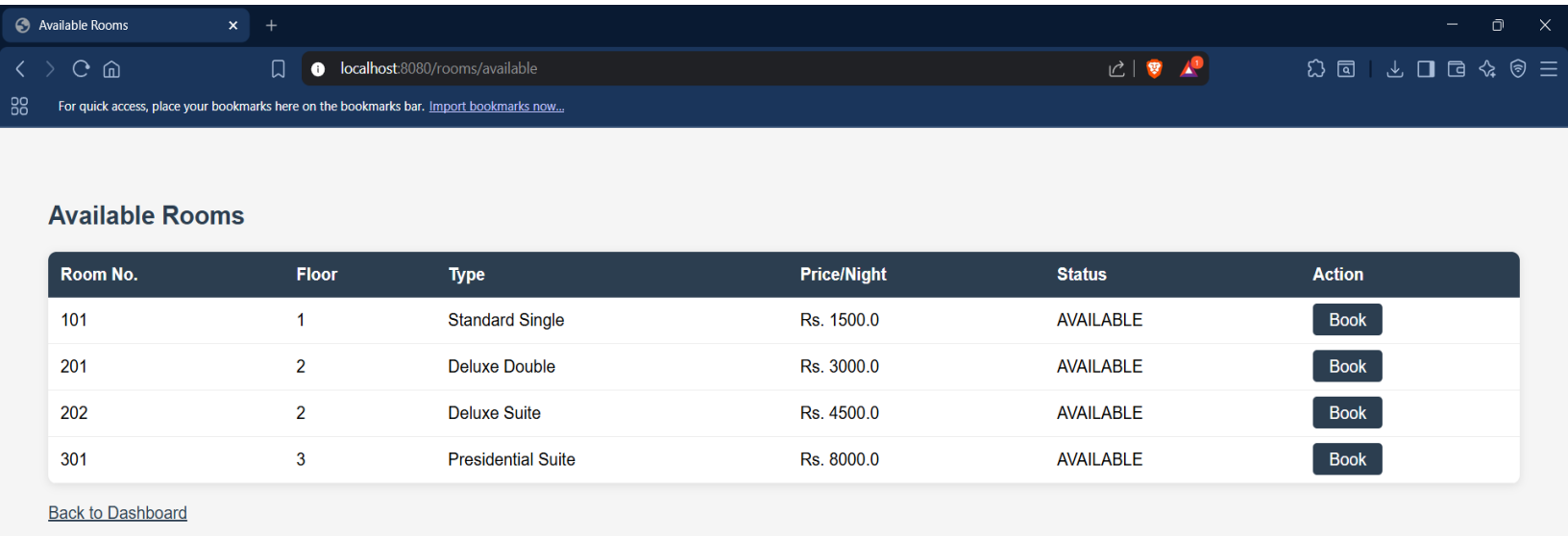
1.1 GUEST LOGIN:



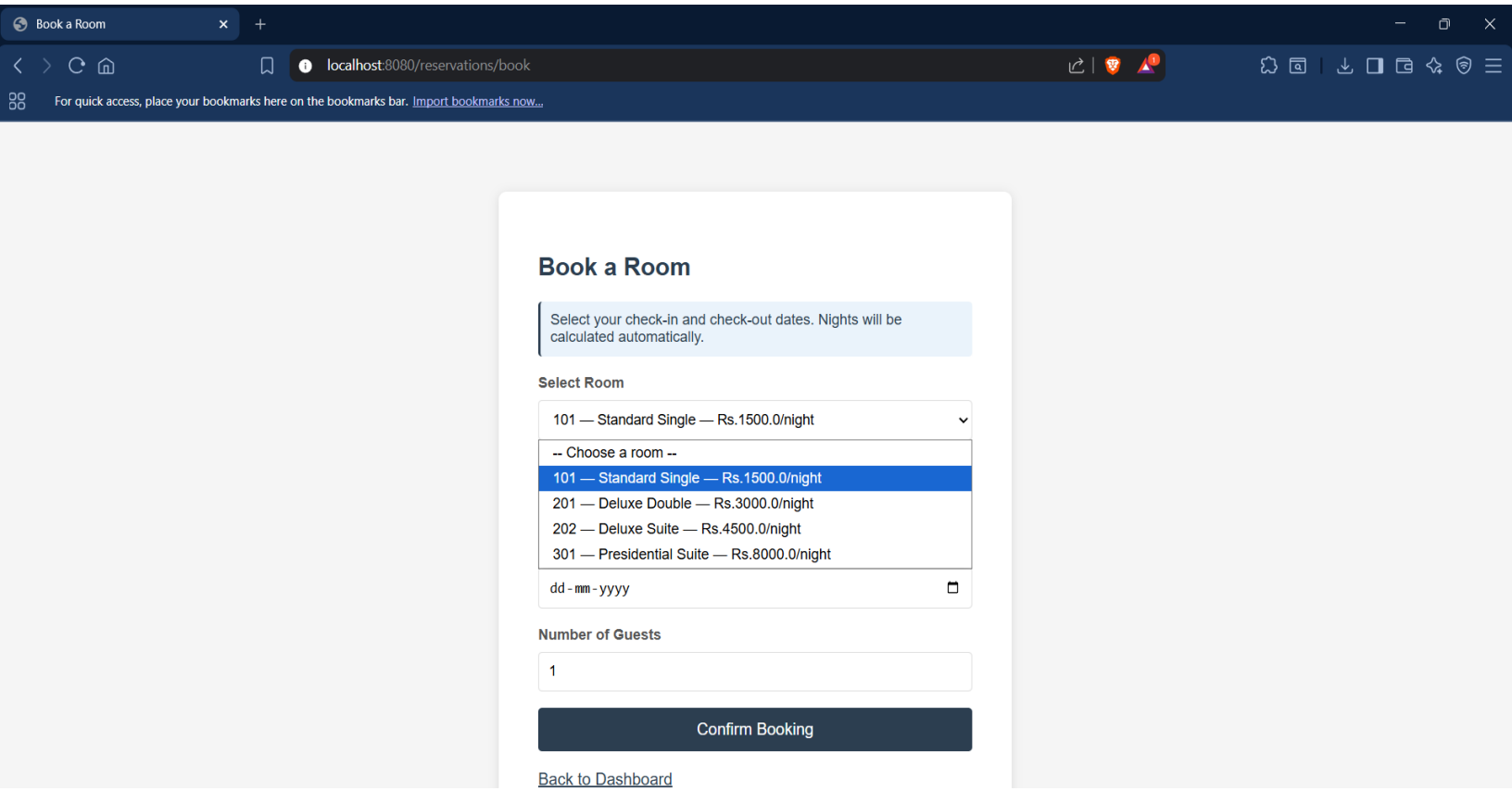

1.2 GUEST DASHBOARD:

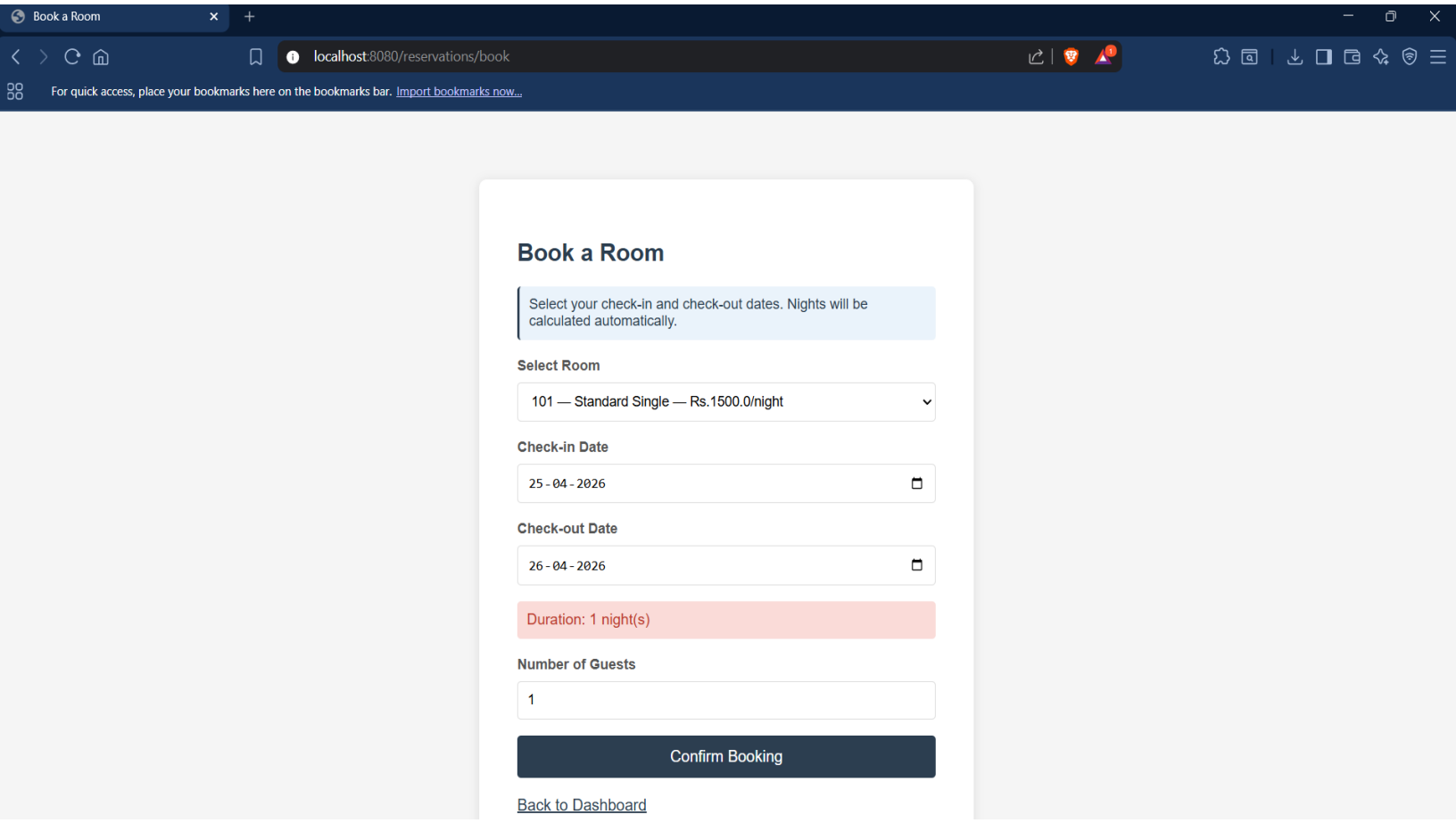


## 1.3 AVAILABLE ROOMS:

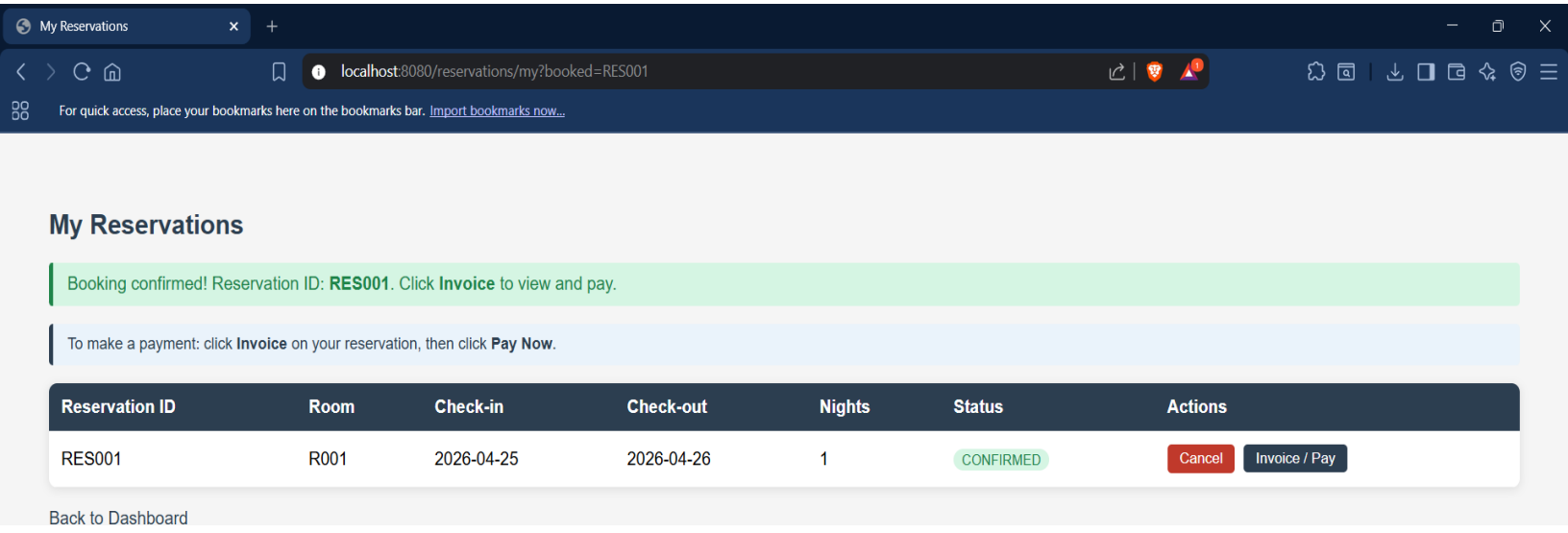



## 1.4 BOOKING A ROOM:

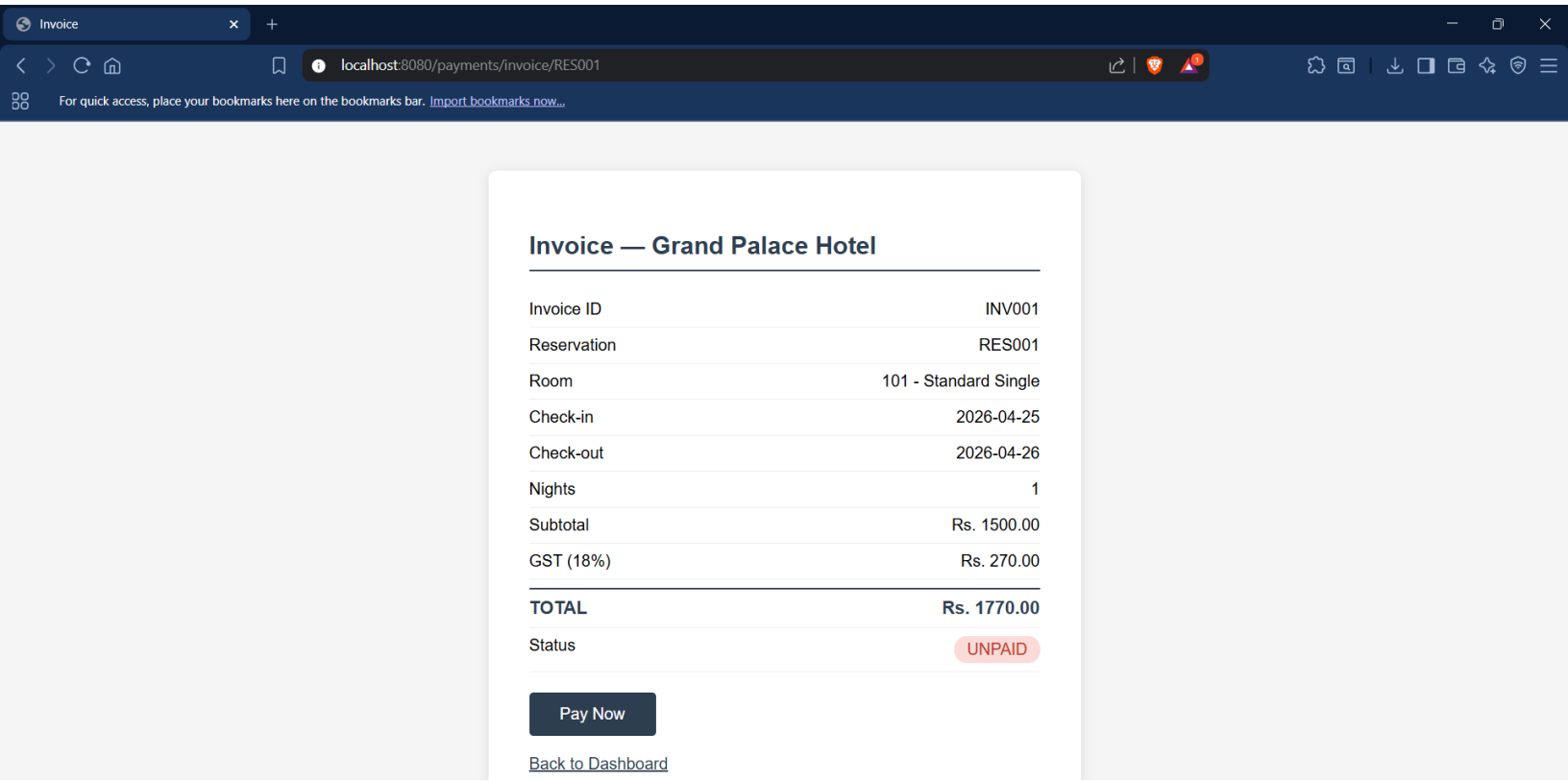




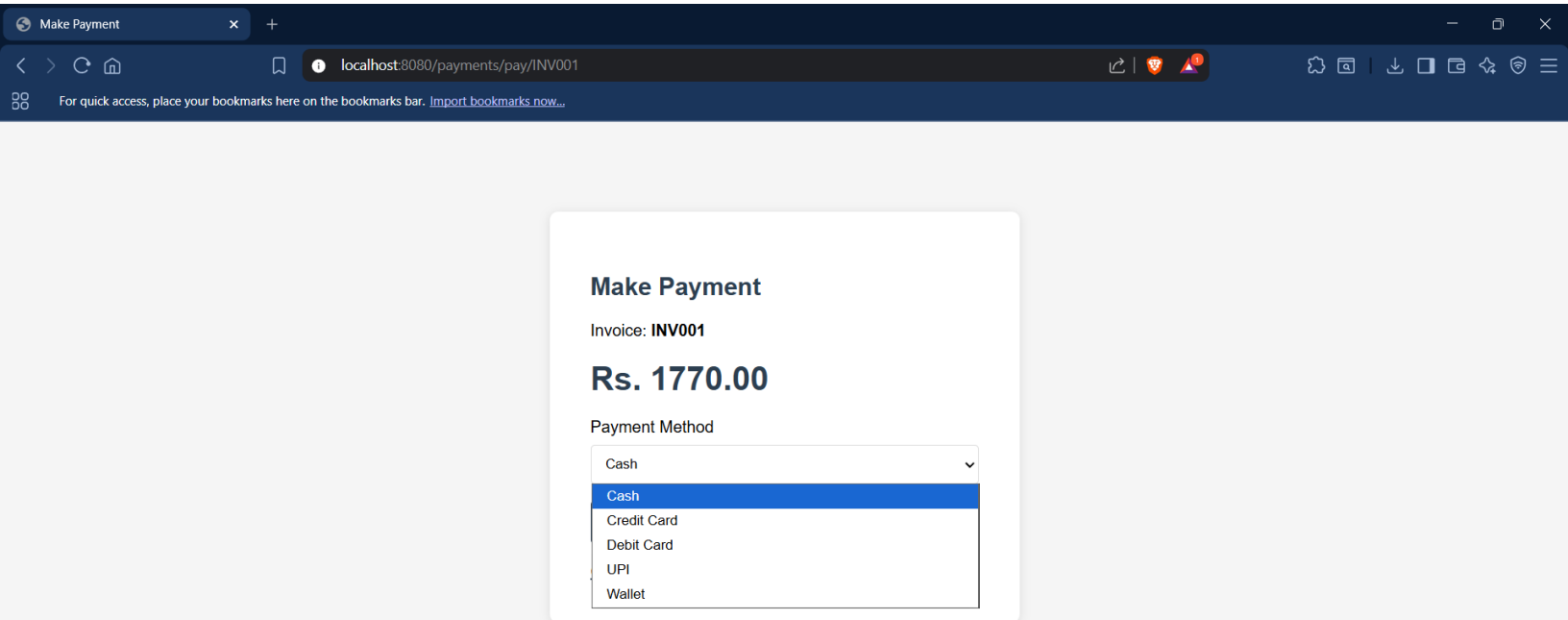

1.5 MY RESERVATIONS:

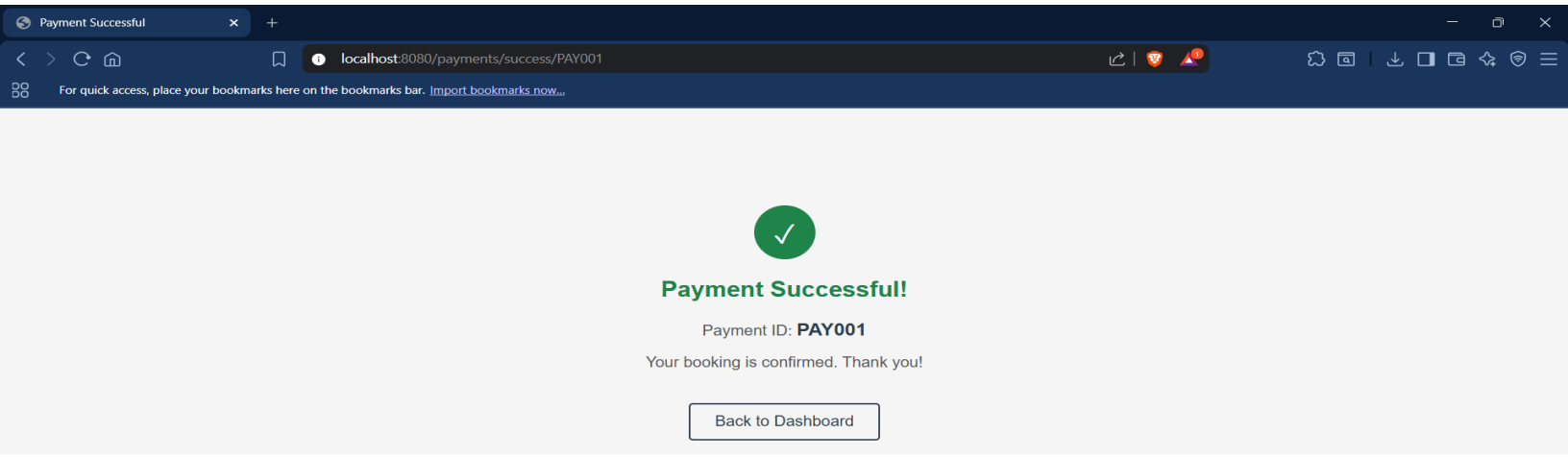


1.6 INVOICE/ PAY:

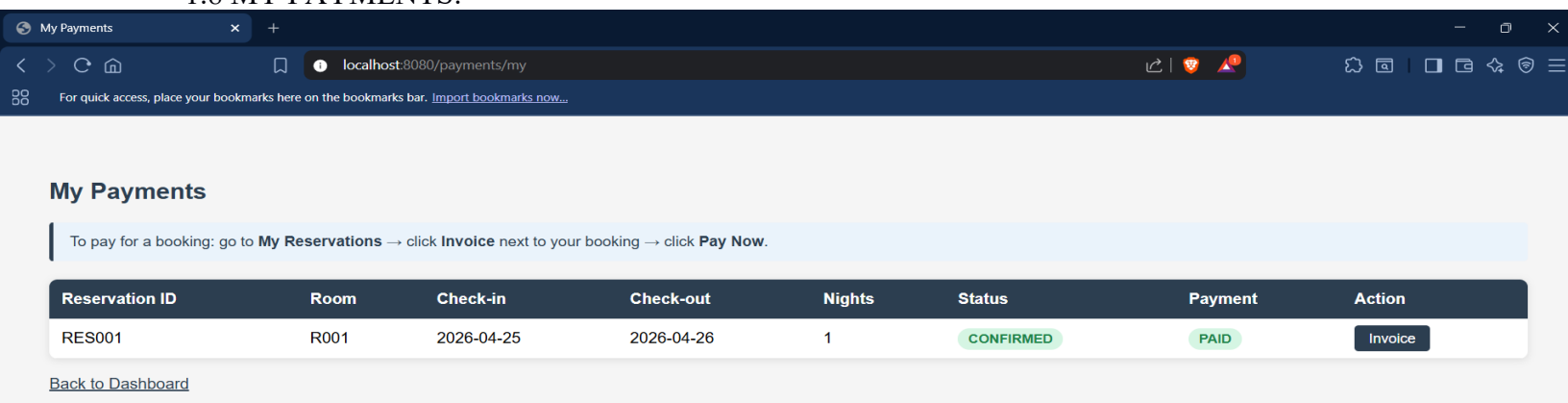



1.7 PAYMENT PAGE:

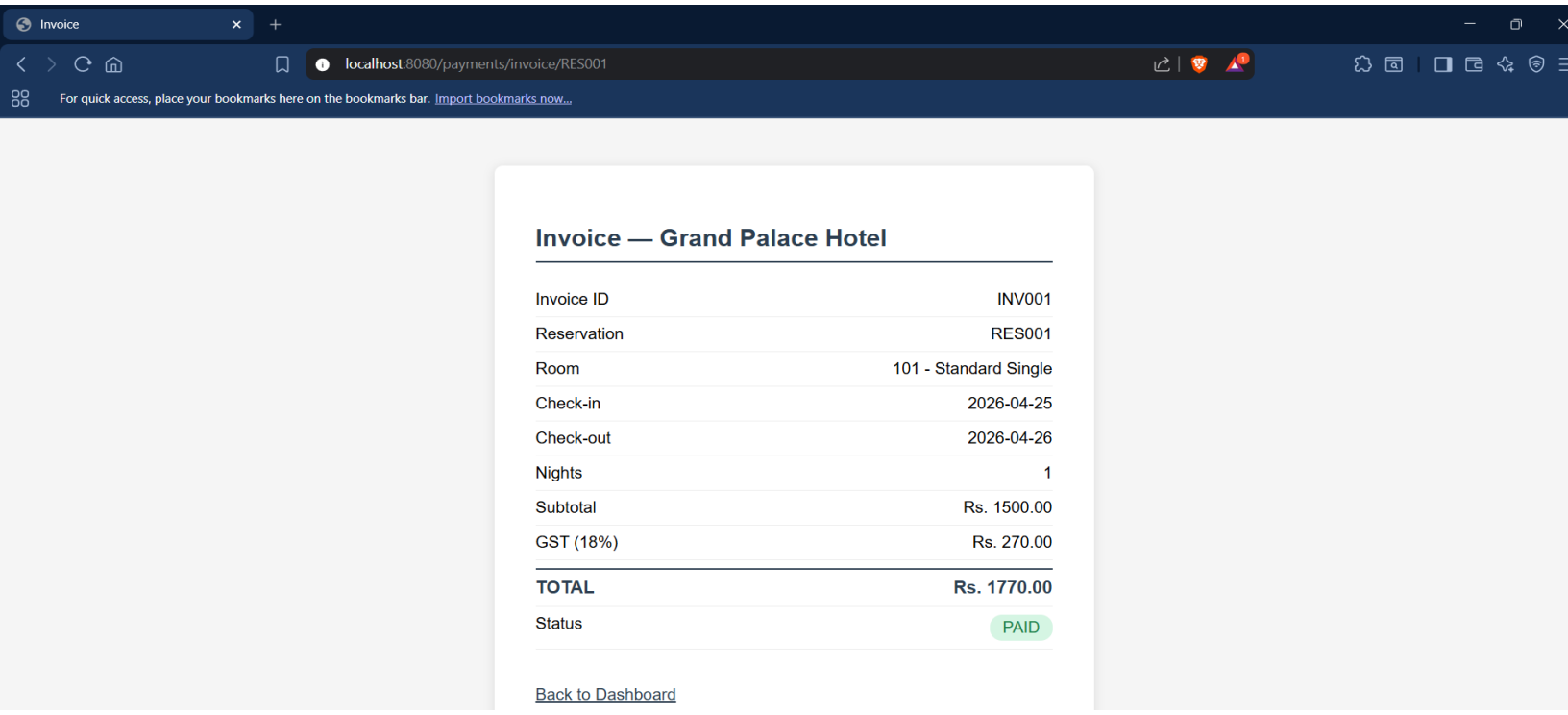




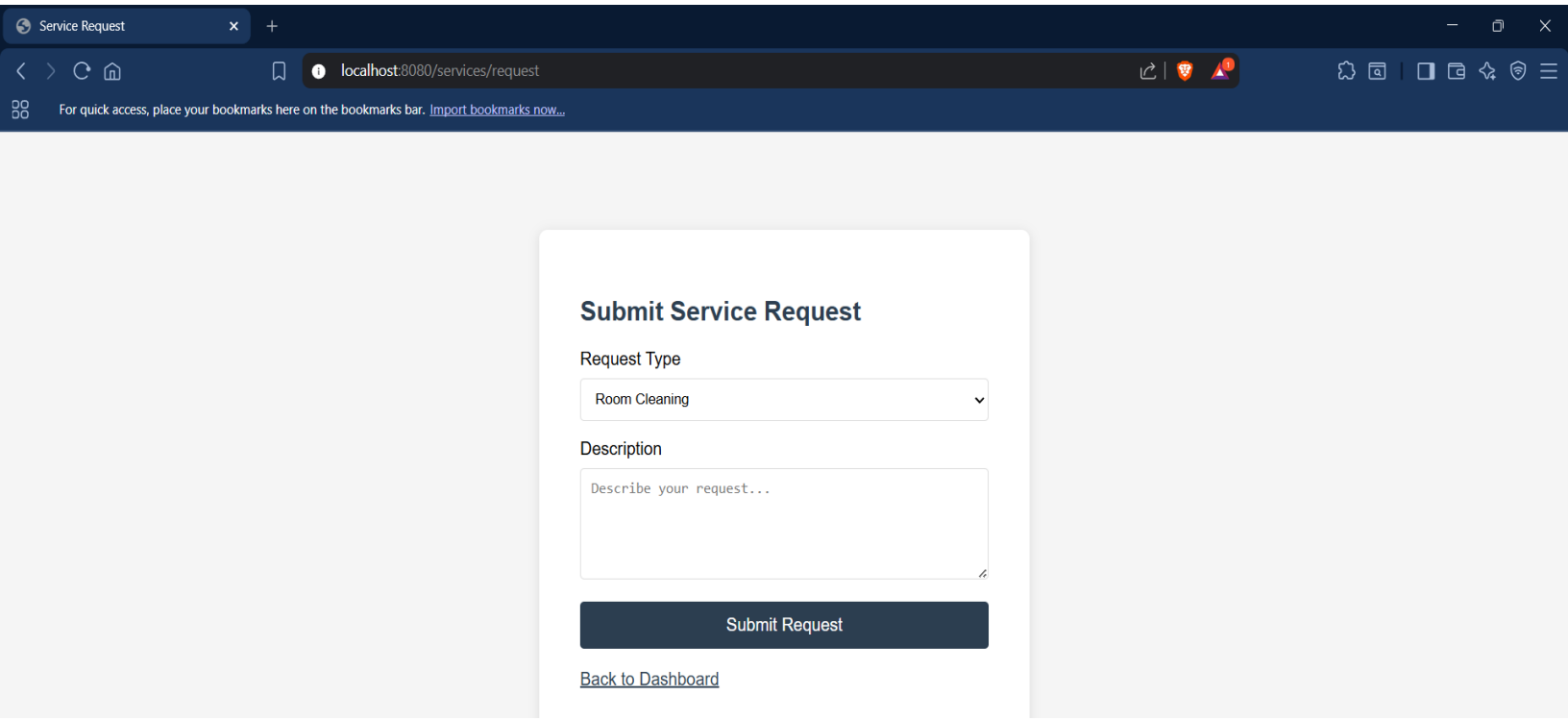

1.8 MY PAYMENTS:

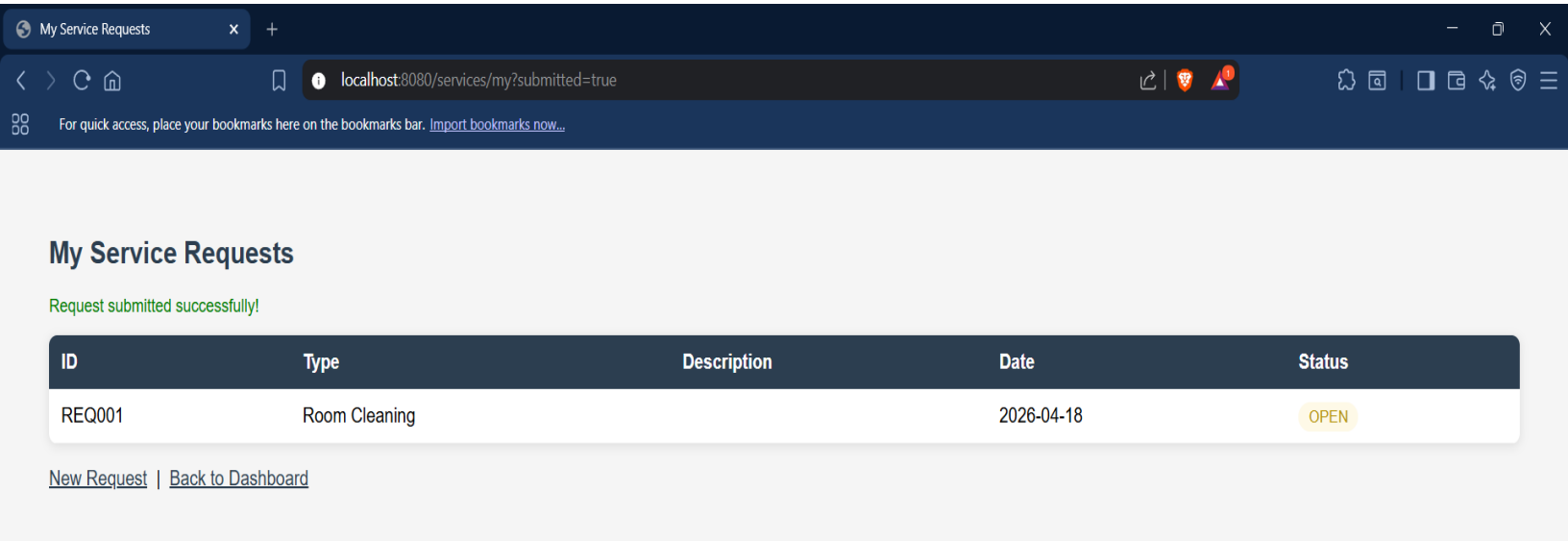



1.9 INVOICE:

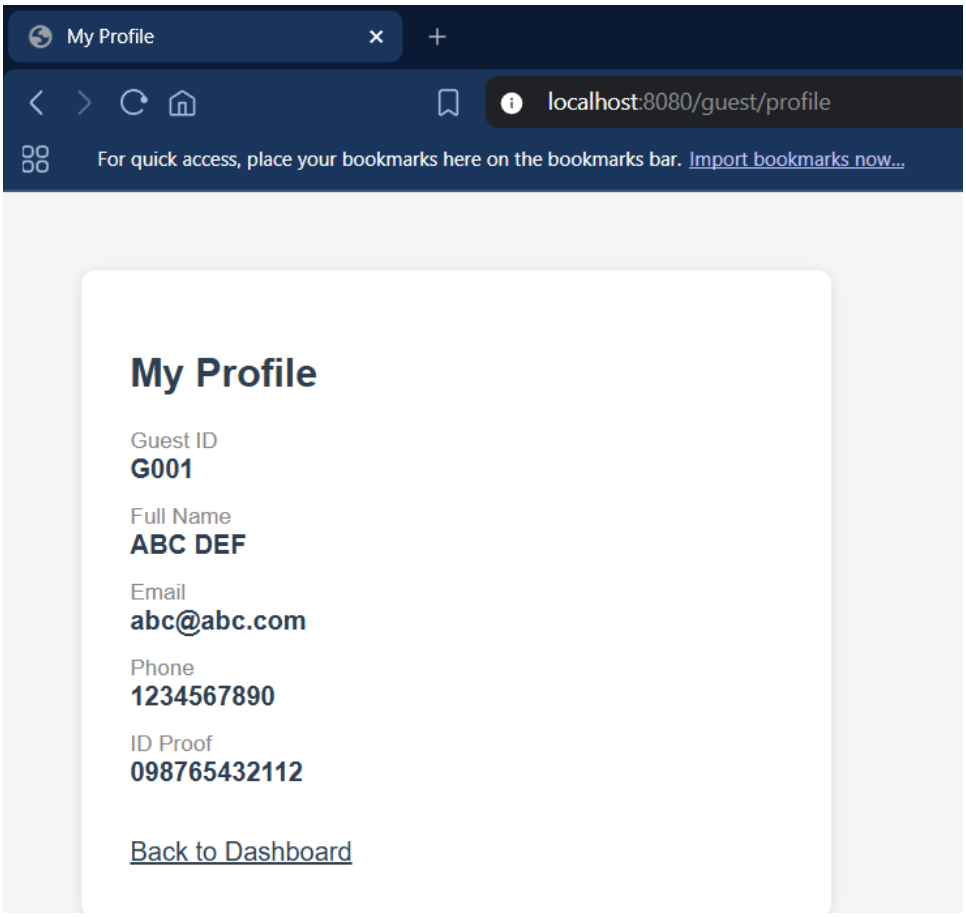


## 1.10 SERVICE REQUEST:



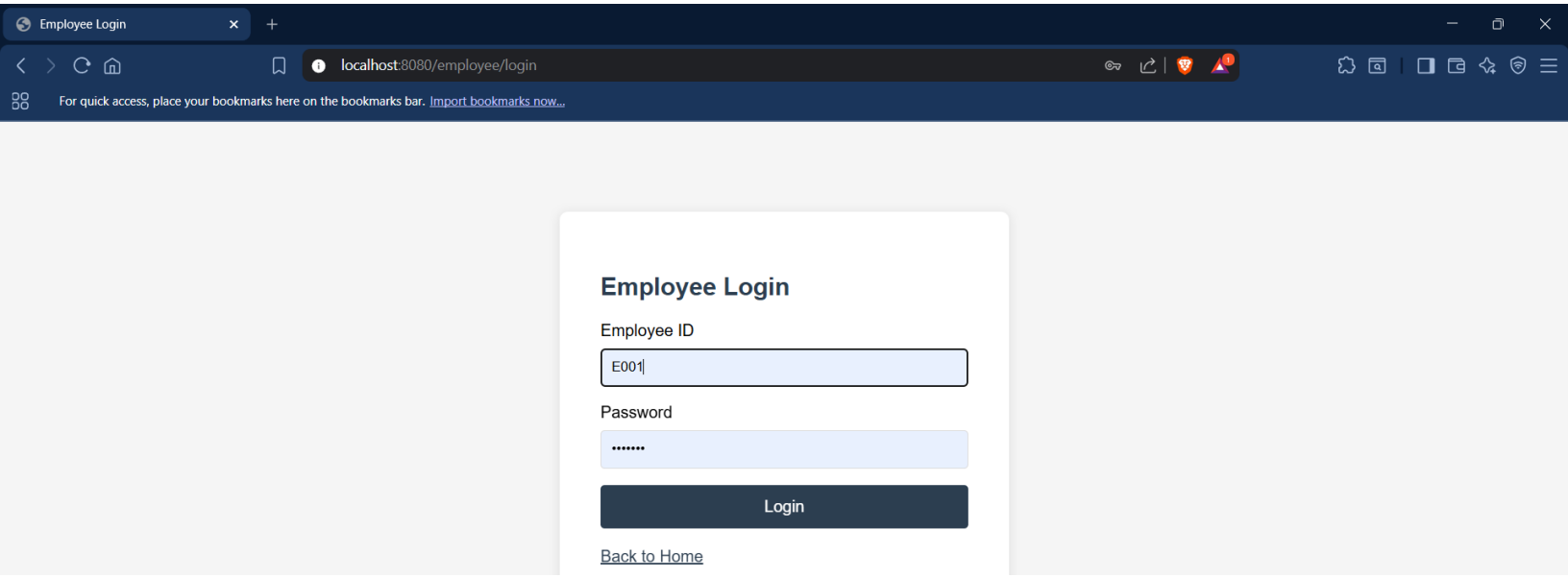



1.11 MY PROFILE:

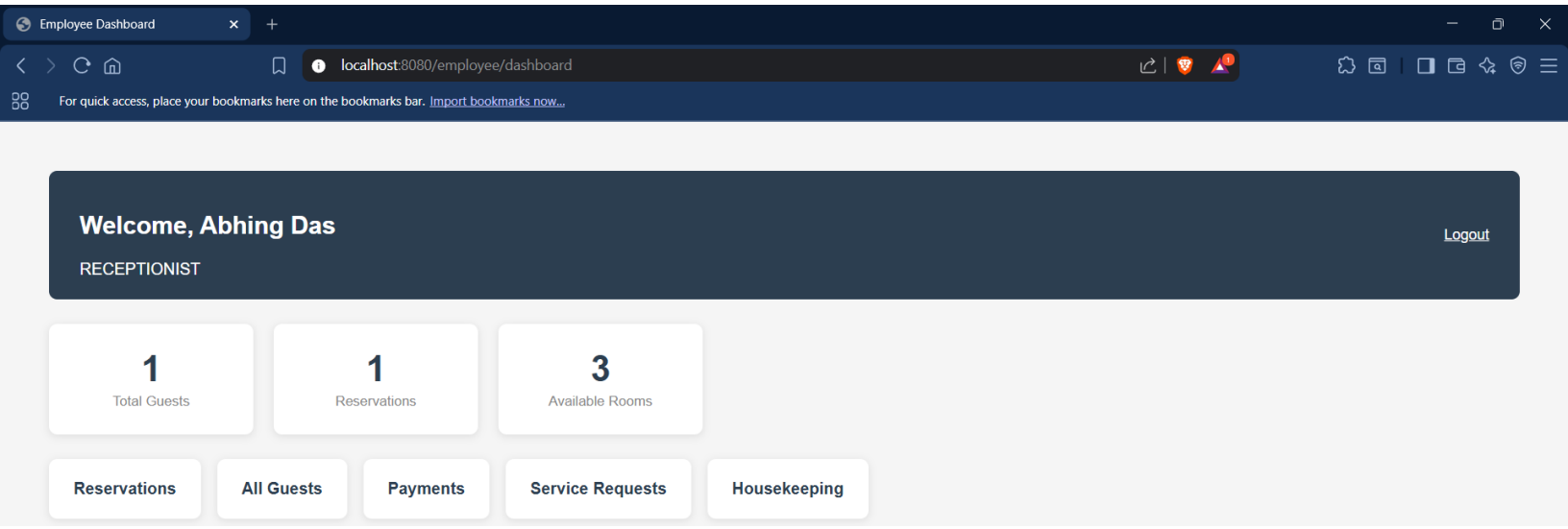



2. EMPLOYEE:

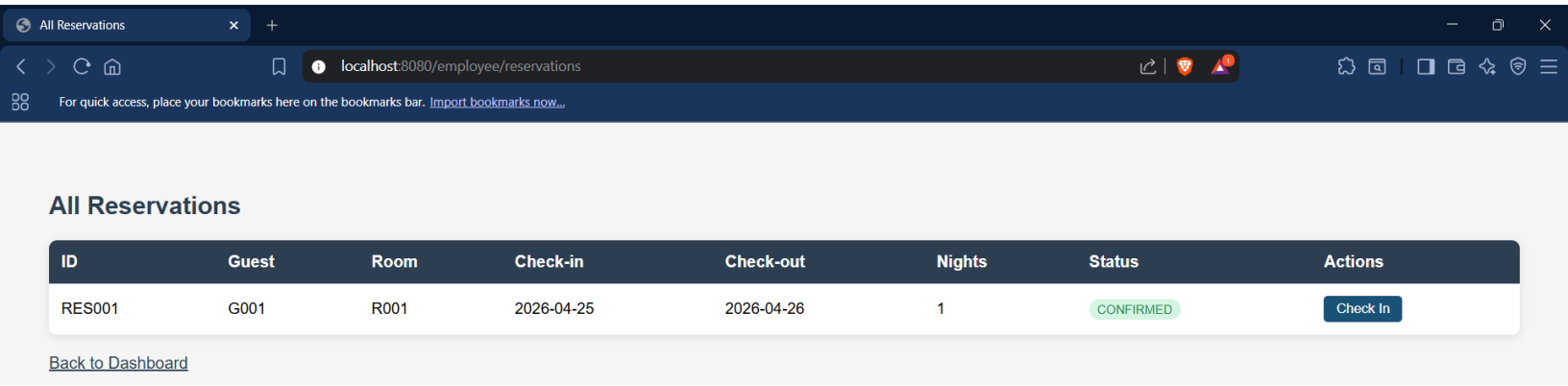
2.1 LOGIN:



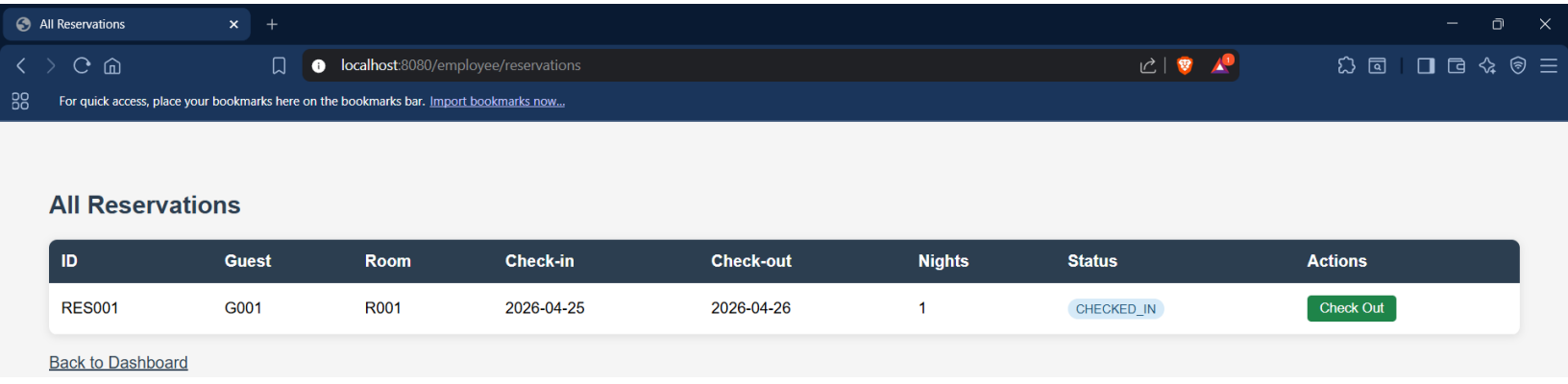
## 2.2 EMPLOYEE DASHBOARD:



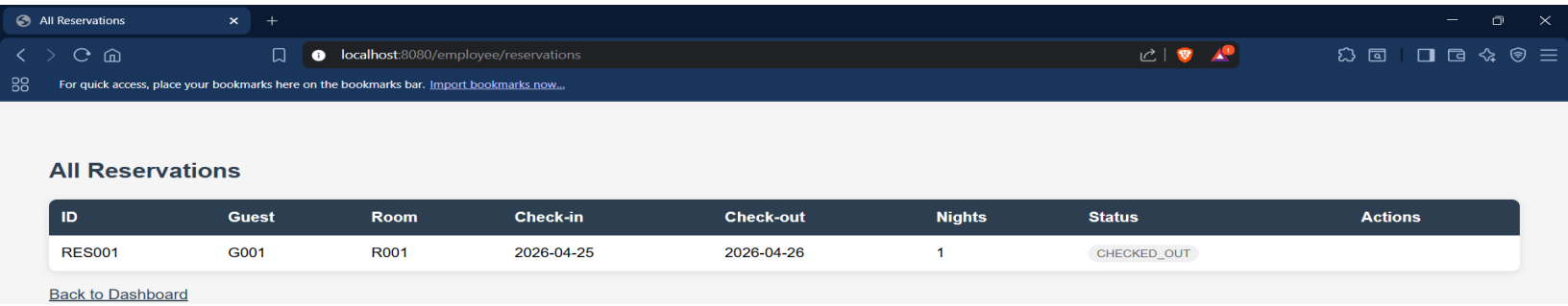

## 2.3 RESERVATIONS:



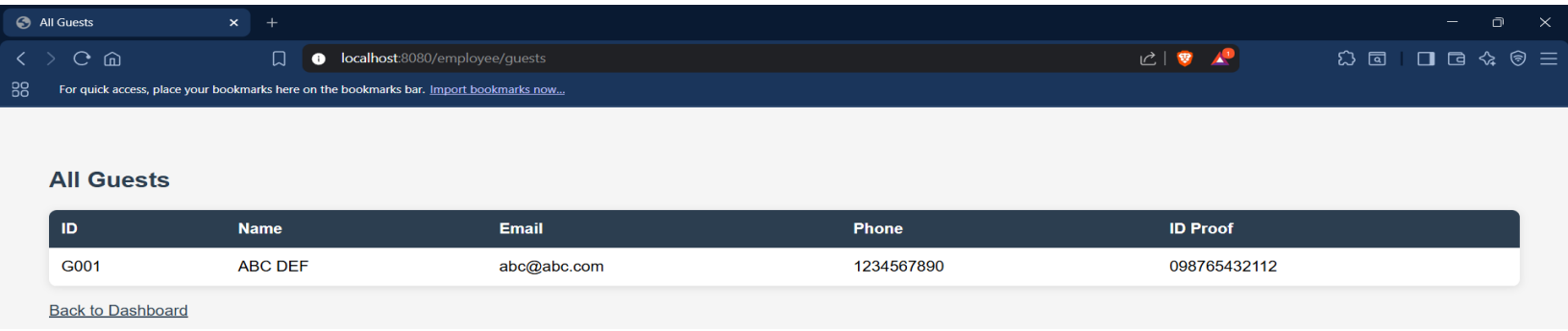

### 2.3.1 CHECKIN ACTION:



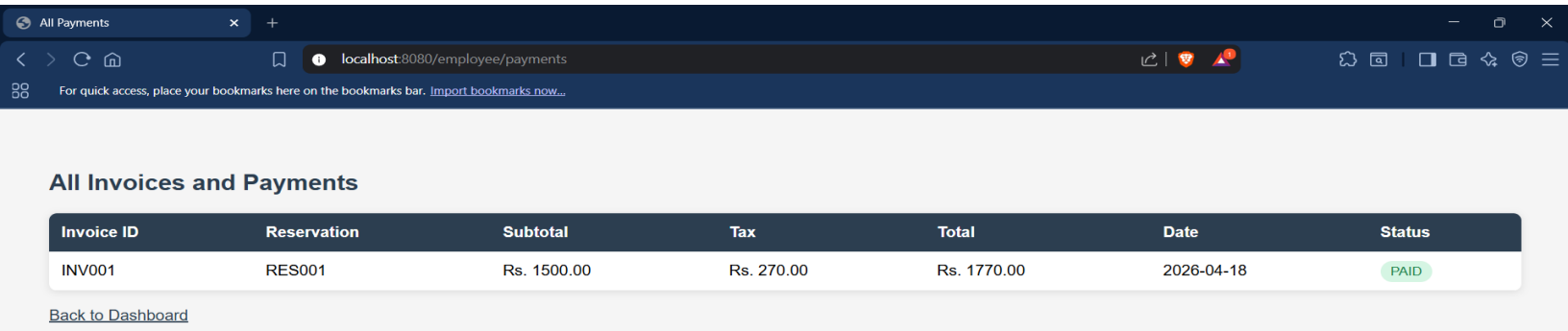
### 2.3.2 CHECKOUT ACTION:



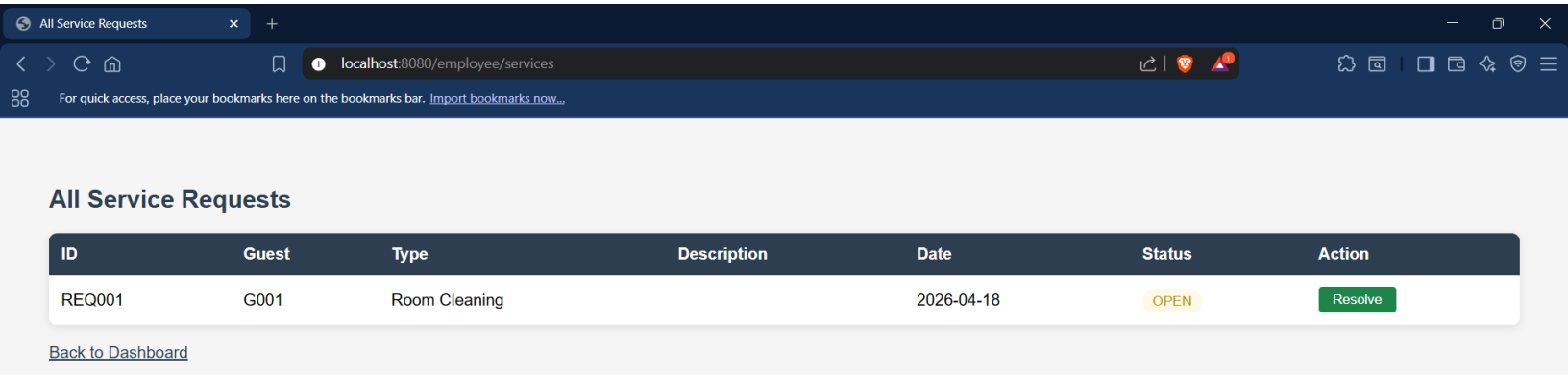

### 2.4 GUEST LIST:



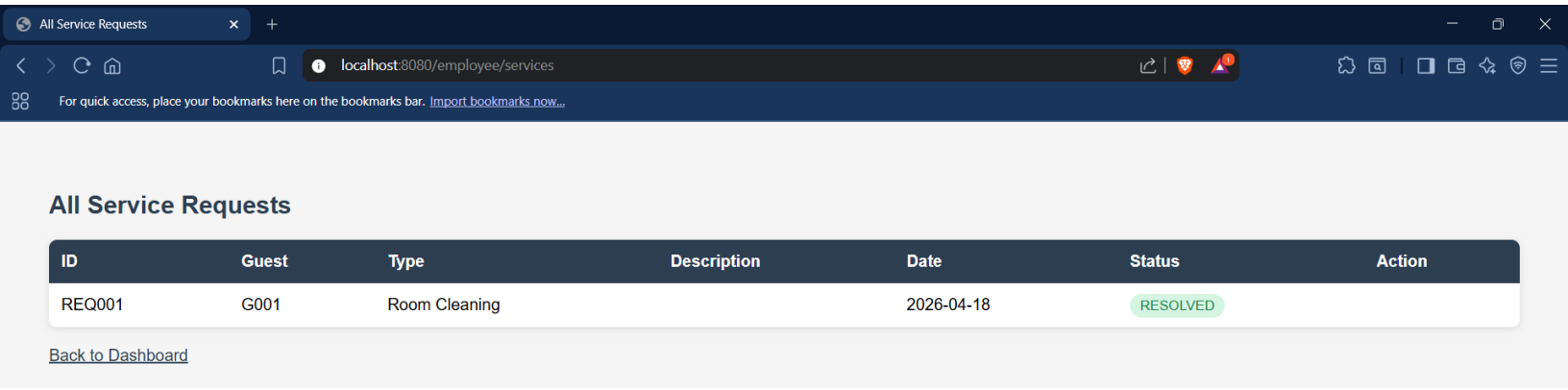

### 2.5 PAYMENT LIST:



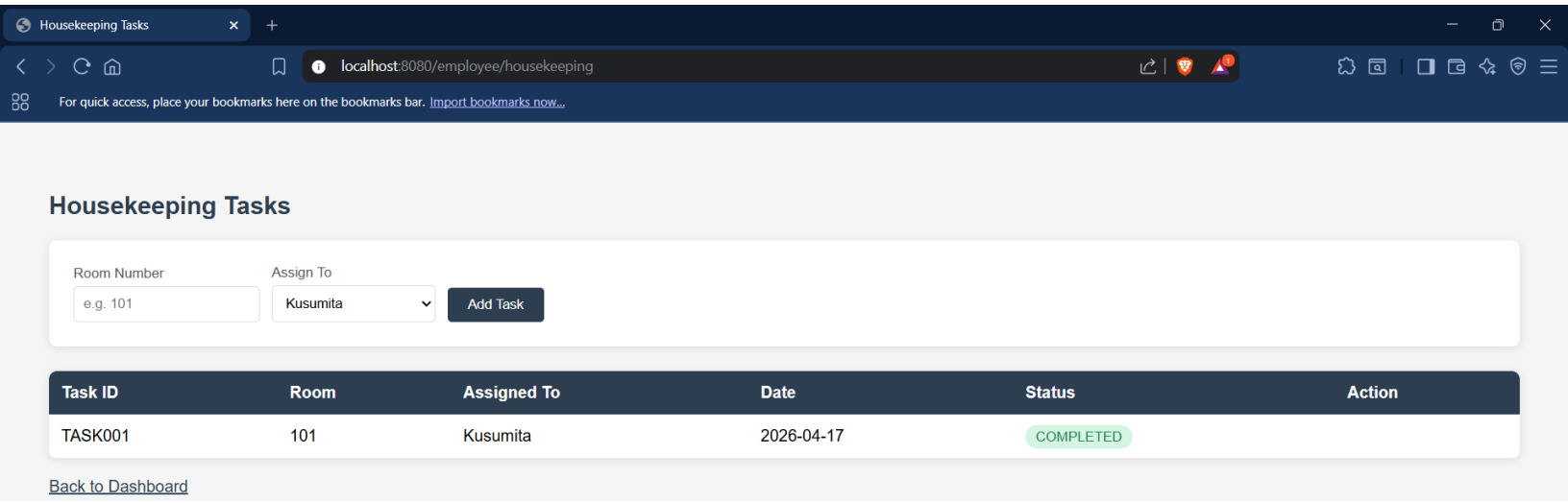

### 2.6 SERVICES REQUEST LIST:



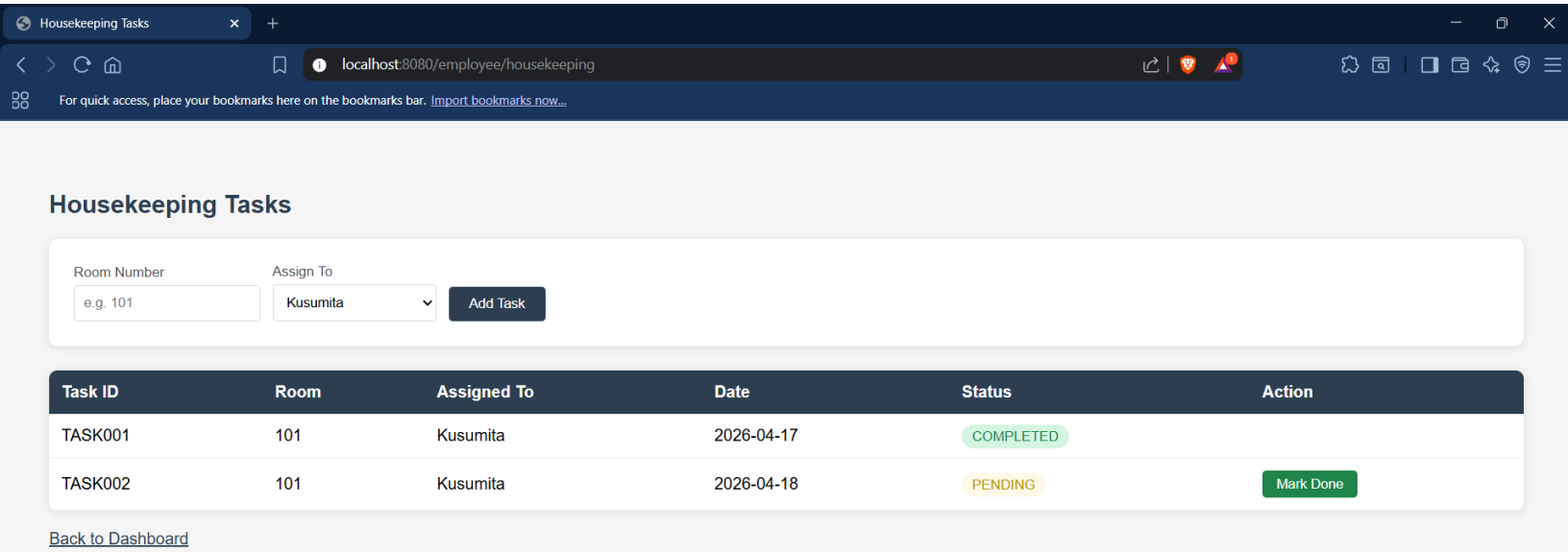
2.6.1 RESOLVE ACTION:



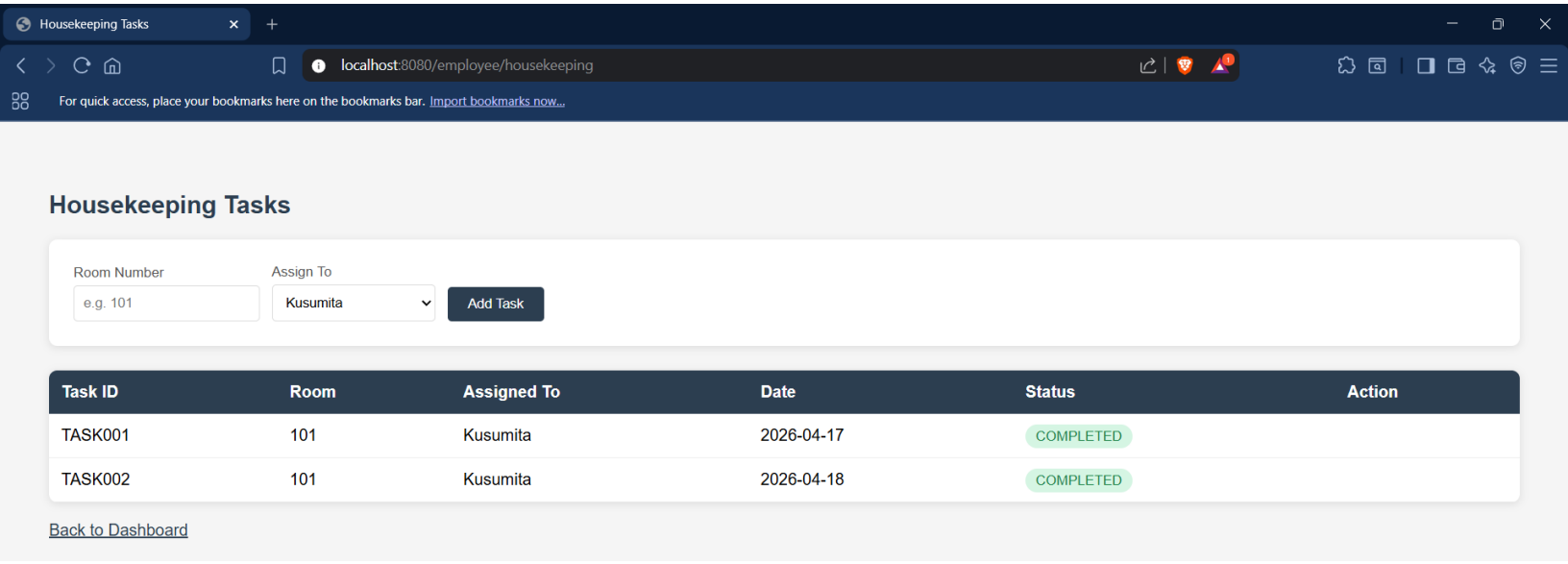

2.7 HOUSEKEEPING:




2.7.1 TASK ASSIGNED:



### 2.7.2 TASK COMPLETED:




Individual contributions of the team members:

| Name | Module worked on |
|---|---|
| **ABHIN G DAS** | Guest Management + Employee Base |
| **AARUSH RYAN LOBO** | Reservation & Room Management |
| **AMOGH VAIDYA** | Billing & Payment |
| **KUSUMITA** | Services & Housekeeping |